```
// ===== FILE: Assets\Editor\SettingsPanelGenerator.cs =====
using UnityEngine;
using UnityEditor;
using UnityEngine.UI;
using System.IO;
#if UNITY_EDITOR
/// <summary>
/// 一键生成设置面板预制体：在 Assets/Resources/SettingsPanel.prefab 创建
  含完整 UI 层级的预制体
/// （遮罩 / 米色卡片 / 标题 / 音乐 · 音效 · 震动开关 / 画质三档 / 语音 / 返回），根挂 Sett
  ingsPanel 组件。
/// 运行时只实例化并显示，组件只查找绑定、不生成 UI。
/// 使用：Unity 菜单 Tools / 生成设置面板预制体（若已存在则覆盖）。
/// </summary>
public class SettingsPanelGenerator
{
  [MenuItem("Tools/生成设置面板预制体")]
  public static void Generate()
  {
    var root = new GameObject("SettingsPanel", typeof(
  RectTransform));
    root.AddComponent<SettingsPanel>();
    var img = root.AddComponent<Image>();
    img.color = new Color(0, 0, 0, 0.7f);
    var lg = root.AddComponent<VerticalLayoutGroup>();
    lg.childAlignment = TextAnchor.MiddleCenter;
    lg.childControlWidth = true; lg.childControlHeight
   = true;
    lg.childForceExpandWidth = false; lg.childForceExp
  andHeight = false;
    lg.spacing = 24;
    var font = Resources.GetBuiltinResource<Font>("Leg
  acyRuntime.ttf");
    MakeText(root.transform, "设置", 46, font, Color.whi
  te);
    var card = MakeGO("SettingsCard", root.transform);
    var cimg = card.AddComponent<Image>();
    cimg.color = new Color(0.93f, 0.89f, 0.80f, 1f);
    var clg = card.AddComponent<VerticalLayoutGroup>()
  ;
    clg.spacing = 26; clg.padding = new RectOffset(36,
   36, 32, 32);
    clg.childAlignment = TextAnchor.MiddleCenter;
    clg.childControlWidth = true; clg.childControlHeig
  ht = true;
    clg.childForceExpandWidth = true; clg.childForceEx
  pandHeight = false;
    var cle = card.AddComponent<LayoutElement>();
    cle.minWidth = 560; cle.preferredWidth = 560; cle.
  preferredHeight = 560;
    BuildToggleRow(card.transform, "Row_音乐", "音乐", fon
  t);
    BuildToggleRow(card.transform, "Row_音效", "音效", fon
  t);
    BuildToggleRow(card.transform, "Row_震动", "震动", fon
  t);
    BuildQualityRow(card.transform, font);
    BuildLangRow(card.transform, font);
    var back = MakeButton(root.transform, "返回", font);
    back.name = "BackBtn";
    if (!Directory.Exists("Assets/Resources/Prefab"))
  Directory.CreateDirectory("Assets/Resources/Prefab");
    string path = "Assets/Resources/Prefab/SettingsPan
  el.prefab";
    PrefabUtility.SaveAsPrefabAsset(root, path);
    UnityEngine.Object.DestroyImmediate(root);
    AssetDatabase.SaveAssets();
```

```
    Debug.Log("[Settings] 已生成含完整 UI 的预制体：" + path);
}
static GameObject MakeGO(string name, Transform parent
)
{
    var go = new GameObject(name, typeof(RectTransform
));
    go.transform.SetParent(parent, false);
    return go;
}
static Text MakeText(Transform parent, string s, int s
ize, Font font, Color color)
{
    var go = MakeGO(s, parent);
    var t = go.AddComponent<Text>();
    t.text = s; t.font = font; t.fontSize = size;
    t.alignment = TextAnchor.MiddleCenter; t.color = c
olor;
    var tr = go.GetComponent<RectTransform>();
    tr.anchorMin = Vector2.zero; tr.anchorMax = Vector
2.one;
    tr.offsetMin = tr.offsetMax = Vector2.zero;
    return t;
}
static Button MakeButton(Transform parent, string labe
l, Font font)
{
    var go = MakeGO(label, parent);
    var img = go.AddComponent<Image>();
    img.color = new Color(0.85f, 0.80f, 0.68f);
    var b = go.AddComponent<Button>();
    b.targetGraphic = img;
    MakeText(go.transform, label, 30, font, new Color(
0.35f, 0.30f, 0.22f));
    return b;
}
static void BuildToggleRow(Transform parent, string ro
wName, string label, Font font)
{
    var row = MakeGO(rowName, parent);
    var rlg = row.AddComponent<HorizontalLayoutGroup>(
);
    rlg.spacing = 16; rlg.childAlignment = TextAnchor.
MiddleLeft;
    rlg.childControlWidth = true; rlg.childControlHeig
ht = true;
    rlg.childForceExpandWidth = false; rlg.childForceE
xpandHeight = true;
    var rle = row.AddComponent<LayoutElement>();
    rle.preferredHeight = 64; rle.flexibleWidth = 1;
    var txt = MakeText(row.transform, label, 34, font,
 new Color(0.35f, 0.30f, 0.22f));
    txt.alignment = TextAnchor.MiddleLeft;
    txt.gameObject.AddComponent<LayoutElement>().prefe
rredWidth = 160;
    var spacer = MakeGO("Spacer", row.transform);
    spacer.AddComponent<LayoutElement>().flexibleWidth
 = 1;
    var track = MakeGO("Track", row.transform);
    track.AddComponent<LayoutElement>().preferredWidth
 = 120;
    track.AddComponent<Image>();
    track.AddComponent<Button>();
    var handle = MakeGO("Handle", track.transform);
    var htr = handle.GetComponent<RectTransform>();
    htr.sizeDelta = new Vector2(44, 44);
    htr.anchorMin = htr.anchorMax = new Vector2(0.5f,
```

```
    0.5f);
        htr.anchoredPosition = new Vector2(30f, 0f);
        handle.AddComponent<Image>().color = new Color(0.9
    7f, 0.96f, 0.90f);
    }
    static void BuildQualityRow(Transform parent, Font fon
    t)
    {
        var row = MakeGO("Row_Quality", parent);
        var rlg = row.AddComponent<HorizontalLayoutGroup>(
    );
        rlg.spacing = 14; rlg.childAlignment = TextAnchor.
    MiddleLeft;
        rlg.childControlWidth = true; rlg.childControlHeig
    ht = true;
        rlg.childForceExpandWidth = false; rlg.childForceE
    xpandHeight = true;
        var rle = row.AddComponent<LayoutElement>();
        rle.preferredHeight = 64; rle.flexibleWidth = 1;
        var txt = MakeText(row.transform, "画面质量", 34, font
    , new Color(0.35f, 0.30f, 0.22f));
        txt.alignment = TextAnchor.MiddleLeft;
        txt.gameObject.AddComponent<LayoutElement>().prefe
    rredWidth = 200;
        var spacer = MakeGO("Spacer", row.transform);
        spacer.AddComponent<LayoutElement>().flexibleWidth
     = 1;
        string[] names = { "低", "中", "高" };
        for (int i = 0; i < 3; i++)
        {
            var b = MakeButton(row.transform, names[i], fo
    nt);
            b.name = "Btn_" + i;
            b.gameObject.AddComponent<LayoutElement>().pre
    ferredWidth = 84;
        }
    }
    static void BuildLangRow(Transform parent, Font font)
    {
        var row = MakeGO("Row_Lang", parent);
        var rlg = row.AddComponent<HorizontalLayoutGroup>(
    );
        rlg.spacing = 14; rlg.childAlignment = TextAnchor.
    MiddleLeft;
        rlg.childControlWidth = true; rlg.childControlHeig
    ht = true;
        rlg.childForceExpandWidth = false; rlg.childForceE
    xpandHeight = true;
        var rle = row.AddComponent<LayoutElement>();
        rle.preferredHeight = 64; rle.flexibleWidth = 1;
        var txt = MakeText(row.transform, "语音", 34, font,
    new Color(0.35f, 0.30f, 0.22f));
        txt.alignment = TextAnchor.MiddleLeft;
        txt.gameObject.AddComponent<LayoutElement>().prefe
    rredWidth = 200;
        var spacer = MakeGO("Spacer", row.transform);
        spacer.AddComponent<LayoutElement>().flexibleWidth
     = 1;
        var b = MakeButton(row.transform, "简体中文    ", font);
        b.name = "LangBtn";
        b.gameObject.AddComponent<LayoutElement>().preferr
    edWidth = 220;
    }
}
#endif
// ===== FILE: Assets\Editor\UiPrefabGenerator.cs =====
using UnityEngine;
```

```
using UnityEditor;
using UnityEngine.UI;
using System.IO;
#if UNITY_EDITOR
/// <summary>
/// 一键生成其余 UI 面板预制体到 Assets/Resources/Prefab/：
///   MenuPanel / PausePanel / ResultPanel / LevelPanel /
   CodexPanel
/// 各预制体含完整 UI 层级（动态内容的面板只含容器，由对应组件运行时填充）。
/// 使用：Unity 菜单 Tools / 生成 UI 预制体。
/// </summary>
public class UiPrefabGenerator
{
   [MenuItem("Tools/生成 UI 预制体")]
   public static void GenerateAll()
   {
      if (!Directory.Exists("Assets/Resources/Prefab"))
   Directory.CreateDirectory("Assets/Resources/Prefab");
      Save(BuildMenu(), "Assets/Resources/Prefab/MenuPan
   el.prefab");
      Save(BuildPause(), "Assets/Resources/Prefab/PauseP
   anel.prefab");
      Save(BuildResult(), "Assets/Resources/Prefab/Resul
   tPanel.prefab");
      Save(BuildLevel(), "Assets/Resources/Prefab/LevelP
   anel.prefab");
      Save(BuildCodex(), "Assets/Resources/Prefab/CodexP
   anel.prefab");
      Save(BuildLevelItem(), "Assets/Resources/Prefab/Le
   velItem.prefab");
      Save(BuildCodexItem(), "Assets/Resources/Prefab/Co
   dexItem.prefab");
      Save(BuildGame(), "Assets/Resources/Prefab/GamePan
   el.prefab");
      AssetDatabase.SaveAssets();
      Debug.Log("[UI] 已生成 Menu/Pause/Result/Level/Codex/
   Game 预制体到 Resources/Prefab");
   }
   static void Save(GameObject root, string path)
   {
      PrefabUtility.SaveAsPrefabAsset(root, path);
      UnityEngine.Object.DestroyImmediate(root);
   }
   static GameObject BuildMenu()
   {
      var root = UiTools.MakePanel("MenuPanel");
      root.AddComponent<MenuPanel>();
      UiTools.AddText(root.transform, "麻将连连看", 64, Color
   .white);
      var b1 = UiTools.AddButton(root.transform, "开始游戏")
   ; b1.name = "Btn_Start";
      var b2 = UiTools.AddButton(root.transform, "选择关卡")
   ; b2.name = "Btn_Levels";
      var b3 = UiTools.AddButton(root.transform, "设置");
   b3.name = "Btn_Settings";
      return root;
   }
   static GameObject BuildPause()
   {
      var root = UiTools.MakePanel("PausePanel");
      root.AddComponent<PausePanel>();
      UiTools.AddText(root.transform, "暂停", 56, Color.wh
   ite);
      var b1 = UiTools.AddButton(root.transform, "继续");
   b1.name = "Btn_Resume";
      var b2 = UiTools.AddButton(root.transform, "重开本关")
   ; b2.name = "Btn_Retry";
```

```
    var b3 = UiTools.AddButton(root.transform, "返回主菜单"
); b3.name = "Btn_Menu";
    return root;
}
static GameObject BuildResult()
{
    var root = UiTools.MakePanel("ResultPanel");
    root.AddComponent<ResultPanel>();
    UiTools.AddText(root.transform, "", 56, Color.whit
e).name = "Title";
    UiTools.AddText(root.transform, "", 32, Color.whit
e).name = "Info";
    var b1 = UiTools.AddButton(root.transform, "下一关");
 b1.name = "Btn_Next";
    var b2 = UiTools.AddButton(root.transform, "重玩");
b2.name = "Btn_Retry";
    var b3 = UiTools.AddButton(root.transform, "主菜单");
 b3.name = "Btn_Menu";
    return root;
}
static GameObject BuildLevel()
{
    var root = UiTools.MakePanel("LevelPanel");
    root.AddComponent<LevelSelectPanel>();
    UiTools.AddText(root.transform, "选择关卡", 56, Color.
white);
    // 关卡列表容器（运行时由 LevelSelectPanel.BuildList 填充）
    var list = UiTools.MakeGO("LevelList", root.transf
orm);
    var llg = list.AddComponent<VerticalLayoutGroup>()
;
    llg.spacing = 12; llg.childAlignment = TextAnchor.
MiddleCenter;
    llg.padding = new RectOffset(40, 40, 10, 10);
    llg.childControlWidth = true; llg.childControlHeig
ht = true;
    llg.childForceExpandWidth = true; llg.childForceEx
pandHeight = false;
    var le = list.AddComponent<LayoutElement>();
    le.minHeight = 200; le.preferredHeight = 600; le.f
lexibleWidth = 1;
    var back = UiTools.AddButton(root.transform, "返回")
; back.name = "Btn_Back";
    return root;
}
static GameObject BuildCodex()
{
    var root = UiTools.MakePanel("CodexPanel");
    root.AddComponent<CodexPanel>();
    var title = UiTools.MakeGO("CodexTitle", root.tran
sform);
    var ttr = title.GetComponent<RectTransform>();
    ttr.anchorMin = new Vector2(0, 1); ttr.anchorMax =
 new Vector2(1, 1); ttr.pivot = new Vector2(0.5f, 1);
    ttr.sizeDelta = new Vector2(0, 80); ttr.anchoredPo
sition = new Vector2(0, -40);
    UiTools.AddText(title.transform, "图鉴", 48, Color.w
hite);
    var scroll = UiTools.MakeGO("CodexScroll", root.tr
ansform);
    var str = scroll.GetComponent<RectTransform>();
    str.anchorMin = new Vector2(0, 0); str.anchorMax =
 new Vector2(1, 1);
    str.offsetMin = new Vector2(30, 110); str.offsetMa
x = new Vector2(-30, -110);
    var sr = scroll.AddComponent<ScrollRect>();
    sr.horizontal = false; sr.vertical = true;
```

```
    var viewport = UiTools.MakeGO("Viewport", scroll.t
ransform);
    var vtr = viewport.GetComponent<RectTransform>();
    vtr.anchorMin = Vector2.zero; vtr.anchorMax = Vect
or2.one; vtr.offsetMin = Vector2.zero; vtr.offsetMax = Vec
tor2.zero;
    var vimg = viewport.AddComponent<Image>(); vimg.co
lor = Color.white;
    viewport.AddComponent<Mask>().showMaskGraphic = tr
ue;
    var content = UiTools.MakeGO("Content", viewport.t
ransform);
    var ctr = content.GetComponent<RectTransform>();
    ctr.anchorMin = new Vector2(0, 1); ctr.anchorMax =
 new Vector2(1, 1); ctr.pivot = new Vector2(0.5f, 1);
    ctr.sizeDelta = new Vector2(0, 0);
    var vlg = content.AddComponent<VerticalLayoutGroup
>();
    vlg.spacing = 8; vlg.padding = new RectOffset(10,
10, 10, 10); vlg.childAlignment = TextAnchor.UpperCenter;
    sr.content = ctr; sr.viewport = vtr;
    var closeRow = UiTools.MakeGO("CloseRow", root.tra
nsform);
    var crtr = closeRow.GetComponent<RectTransform>();
    crtr.anchorMin = new Vector2(0, 0); crtr.anchorMax
 = new Vector2(1, 0); crtr.pivot = new Vector2(0.5f, 0);
    crtr.sizeDelta = new Vector2(0, 100); crtr.anchore
dPosition = new Vector2(0, 50);
    var cb = UiTools.AddButton(closeRow.transform, "关闭
"); cb.name = "CloseBtn";
    return root;
}
// 关卡列表项：根即 Button（背景 Image）+ 子 Text，运行时按数据 Bind
static GameObject BuildLevelItem()
{
    var root = new GameObject("LevelItem", typeof(Rect
Transform));
    var img = root.AddComponent<Image>();
    img.color = new Color(0.20f, 0.50f, 0.82f);
    var btn = root.AddComponent<Button>();
    btn.targetGraphic = img;
    var le = root.AddComponent<LayoutElement>();
    le.minHeight = 64; le.preferredHeight = 64; le.fle
xibleWidth = 1;
    root.AddComponent<LevelItem>();
    UiTools.AddText(root.transform, "关卡", 26, Color.wh
ite);
    return root;
}
// 图鉴行项：根（HLG）+ Icon(Image) + Txt(Text) + Color(Image)
，运行时按数据 Bind
static GameObject BuildCodexItem()
{
    var root = new GameObject("CodexItem", typeof(Rect
Transform));
    root.GetComponent<RectTransform>().sizeDelta = new
 Vector2(0, 64);
    var hlg = root.AddComponent<HorizontalLayoutGroup>
();
    hlg.spacing = 12; hlg.childAlignment = TextAnchor.
MiddleLeft;
    hlg.childForceExpandWidth = false; hlg.childForceE
xpandHeight = false;
    root.AddComponent<CodexItem>();
    var icon = UiTools.MakeGO("Icon", root.transform);
    icon.GetComponent<RectTransform>().sizeDelta = new
 Vector2(48, 48);
```

```
    icon.AddComponent<Image>().preserveAspect = true;
    var txt = UiTools.MakeGO("Txt", root.transform);
    txt.GetComponent<RectTransform>().sizeDelta = new
Vector2(200, 48);
    var t = txt.AddComponent<Text>();
    t.text = "x0  Lv0"; t.fontSize = 24; t.alignment =
 TextAnchor.MiddleLeft;
    if (UiTools.LoadFont() != null) t.font = UiTools.L
oadFont();
    var col = UiTools.MakeGO("Color", root.transform);
    col.GetComponent<RectTransform>().sizeDelta = new
Vector2(48, 48);
    col.AddComponent<Image>().preserveAspect = true;
    return root;
}
// 游戏界面（HUD）：顶栏 + 底栏 + 棋盘容器 + Toast，按钮/文本命名匹配 GameCont
roller 引用
[MenuItem("Tools/生成游戏界面预制体")]
public static void GenerateGame()
{
    if (!Directory.Exists("Assets/Resources/Prefab"))
Directory.CreateDirectory("Assets/Resources/Prefab");
    Save(BuildGame(), "Assets/Resources/Prefab/GamePan
el.prefab");
    AssetDatabase.SaveAssets();
    Debug.Log("[UI] 已生成 GamePanel.prefab 到 Resources/P
refab");
}
static GameObject BuildGame()
{
    var root = UiTools.MakePanel("GamePanel");
    root.AddComponent<GamePanel>();
    var gp = root.GetComponent<GamePanel>();
    // 顶栏：分数 / 最高分 / 局数 / 暂停 / 图鉴
    var top = UiTools.MakeGO("TopBar", root.transform)
;
    var ttr = top.GetComponent<RectTransform>();
    ttr.anchorMin = new Vector2(0, 1); ttr.anchorMax =
 new Vector2(1, 1); ttr.pivot = new Vector2(0.5f, 1);
    ttr.sizeDelta = new Vector2(0, 80); ttr.anchoredPo
sition = new Vector2(0, -40);
    var topLg = top.AddComponent<HorizontalLayoutGroup
>();
    topLg.padding = new RectOffset(16, 16, 8, 8); topL
g.spacing = 24; topLg.childAlignment = TextAnchor.MiddleCe
nter;
    var score = UiTools.AddText(top.transform, "分数 0",
 28, Color.white); score.name = "ScoreLabel";
    var best = UiTools.AddText(top.transform, "最高 0",
28, Color.white); best.name = "BestLabel";
    var round = UiTools.AddText(top.transform, "第 1 局"
, 28, Color.white); round.name = "RoundLabel";
    foreach (var t in new[] { score, best, round }) {
var le = t.gameObject.AddComponent<LayoutElement>(); le.fl
exibleWidth = 1; }
    var pause = UiTools.AddButton(top.transform, "暂停")
; pause.name = "PauseBtn";
    var codex = UiTools.AddButton(top.transform, "图鉴")
; codex.name = "CodexBtn";
    foreach (var b in new[] { pause, codex }) { var le
 = b.gameObject.AddComponent<LayoutElement>(); le.minHeigh
t = 64; le.flexibleWidth = 1; }
    // 底栏：提示 / 洗牌 / 重开 / 层偏移滑条（调试）
    var bottom = UiTools.MakeGO("BottomBar", root.tran
sform);
    var btr = bottom.GetComponent<RectTransform>();
    btr.anchorMin = new Vector2(0, 0); btr.anchorMax =
```

```
 new Vector2(1, 0); btr.pivot = new Vector2(0.5f, 0);
   btr.sizeDelta = new Vector2(0, 96); btr.anchoredPo
sition = new Vector2(0, 48);
   var botLg = bottom.AddComponent<HorizontalLayoutGr
oup>();
   botLg.padding = new RectOffset(16, 16, 8, 8); botL
g.spacing = 16; botLg.childAlignment = TextAnchor.MiddleCe
nter; botLg.childForceExpandWidth = true;
   var hint = UiTools.AddButton(bottom.transform, "提示
(5)"); hint.name = "HintBtn";
   var shuffle = UiTools.AddButton(bottom.transform,
"洗牌"); shuffle.name = "ShuffleBtn";
   var restart = UiTools.AddButton(bottom.transform,
"重开"); restart.name = "RestartBtn";
   foreach (var b in new[] { hint, shuffle, restart }
) { var le = b.gameObject.AddComponent<LayoutElement>(); l
e.flexibleWidth = 1; le.minHeight = 64; }
   var xLabel = UiTools.AddText(bottom.transform, "X层
偏移", 22, Color.white); xLabel.name = "XLabel";
   var xs = UiTools.MakeGO("XOffsetSlider", bottom.tr
ansform); var xsSlider = xs.AddComponent<Slider>(); xsSlid
er.minValue = -40; xsSlider.maxValue = 40; xsSlider.value
= 0;
   var xle = xs.AddComponent<LayoutElement>(); xle.mi
nHeight = 36; xle.preferredWidth = 240; xle.flexibleWidth
= 1;
   var yLabel = UiTools.AddText(bottom.transform, "Y层
偏移", 22, Color.white); yLabel.name = "YLabel";
   var ys = UiTools.MakeGO("YOffsetSlider", bottom.tr
ansform); var ysSlider = ys.AddComponent<Slider>(); ysSlid
er.minValue = -40; ysSlider.maxValue = 40; ysSlider.value
= 0;
   var yle = ys.AddComponent<LayoutElement>(); yle.mi
nHeight = 36; yle.preferredWidth = 240; yle.flexibleWidth
= 1;
   // 棋盘容器
   var br = UiTools.MakeGO("BoardRoot", root.transfor
m);
   var brtr = br.GetComponent<RectTransform>();
   brtr.anchorMin = brtr.anchorMax = new Vector2(0.5f
, 0.5f); brtr.pivot = new Vector2(0.5f, 0.5f);
   // 居中提示文字
   var toast = UiTools.MakeGO("Toast", root.transform
);
   var tts = toast.GetComponent<RectTransform>();
   tts.anchorMin = tts.anchorMax = new Vector2(0.5f,
0.5f); tts.pivot = new Vector2(0.5f, 0.5f); tts.sizeDelta
= new Vector2(420, 90);
   var toastText = UiTools.AddText(toast.transform, "
", 34, Color.white); toastText.name = "ToastText";
   // 绑定到 GamePanel 组件字段（SaveAsPrefabAsset 会保存这些引用）
   gp.topBar = top.GetComponent<RectTransform>();
   gp.bottomBar = bottom.GetComponent<RectTransform>(
);
   gp.boardRoot = br.GetComponent<RectTransform>();
   gp.toast = toast.GetComponent<RectTransform>();
   gp.scoreLabel = score; gp.bestLabel = best; gp.rou
ndLabel = round; gp.toastText = toastText;
   gp.hintBtn = hint; gp.shuffleBtn = shuffle; gp.res
tartBtn = restart;
   gp.pauseBtn = pause; gp.codexBtn = codex;
   gp.xOffsetSlider = xsSlider; gp.yOffsetSlider = ys
Slider;
   gp.xLabel = xLabel; gp.yLabel = yLabel;
   return root;
}
// 只生成 Item 预制体（关卡项 / 图鉴行），不触碰面板预制体，避免覆盖你在编辑器里已调好的面板
```

```
    [MenuItem("Tools/生成 Item 预制体")]
    public static void GenerateItems()
    {
        if (!Directory.Exists("Assets/Resources/Prefab"))
    Directory.CreateDirectory("Assets/Resources/Prefab");
        Save(BuildLevelItem(), "Assets/Resources/Prefab/Le
    velItem.prefab");
        Save(BuildCodexItem(), "Assets/Resources/Prefab/Co
    dexItem.prefab");
        AssetDatabase.SaveAssets();
        Debug.Log("[UI] 已生成 LevelItem/CodexItem 预制体到 Resou
    rces/Prefab（未覆盖面板）");
    }
}
#endif
// ===== FILE: Assets\Scripts\Core\GameBoard.cs =====
using System;
using System.Collections.Generic;
using UnityEngine;
/// <summary>
/// 麻将对对碰核心规则（Godot game_board.gd 的 C# 移植）。
/// 自由牌 = 上方无牌压住，且左右两侧不同时被占（至少一侧开放）。
/// </summary>
public class GameBoard
{
    private Dictionary<string, TileGeo> _geo = new Diction
    ary<string, TileGeo>();
    private Dictionary<string, int> _faces = new Dictionar
    y<string, int>();
    private Dictionary<string, bool> _present = new Dictio
    nary<string, bool>();
    private string _selected = "";
    public event Action<string, bool> OnSelectionChanged;
    public event Action<string, string> OnMatched;
    public event Action<string> OnBlocked;
    public event Action OnWin;
    public event Action OnDeadlock;
    public void Setup(Dictionary<string, TileGeo> geometry
    , Dictionary<string, int> faces)
    {
        _geo = geometry;
        _faces = new Dictionary<string, int>(faces);
        _present = new Dictionary<string, bool>();
        foreach (var id in faces.Keys) _present[id] = true
    ;
        _selected = "";
    }
    public bool IsFree(string id)
    {
        if (!_present.ContainsKey(id) || !_present[id]) re
    turn false;
        var g = _geo[id];
        foreach (var a in g.above) if (_present[a]) return
     false;
        bool lb = false, rb = false;
        foreach (var l in g.left) if (_present[l]) { lb =
    true; break; }
        foreach (var r in g.right) if (_present[r]) { rb =
     true; break; }
        return !(lb && rb);
    }
    public void Click(string id)
    {
        if (!_present.ContainsKey(id) || !_present[id]) re
    turn;
        if (!IsFree(id)) { OnBlocked?.Invoke(id); return;
    }
```

```
    if (_selected == "")
    {
        _selected = id;
        OnSelectionChanged?.Invoke(id, true);
        return;
    }
    if (_selected == id)
    {
        _selected = "";
        OnSelectionChanged?.Invoke(id, false);
        return;
    }
    if (_faces[_selected] == _faces[id])
    {
        string a = _selected;
        _selected = "";
        OnSelectionChanged?.Invoke(a, false);
        _present[a] = false;
        _present[id] = false;
        OnMatched?.Invoke(a, id);
        if (IsWin()) OnWin?.Invoke();
        else if (!HasValidMove()) OnDeadlock?.Invoke()
;
    }
    else
    {
        OnSelectionChanged?.Invoke(_selected, false);
        _selected = id;
        OnSelectionChanged?.Invoke(id, true);
    }
}
public bool IsWin()
{
    foreach (var kv in _present) if (kv.Value) return
false;
    return true;
}
public bool HasValidMove()
{
    var seen = new HashSet<int>();
    foreach (var id in _present.Keys)
    {
        if (!_present[id]) continue;
        if (!IsFree(id)) continue;
        int f = _faces[id];
        if (seen.Contains(f)) return true;
        seen.Add(f);
    }
    return false;
}
public int Remaining()
{
    int n = 0;
    foreach (var kv in _present) if (kv.Value) n++;
    return n;
}
public List<string> PresentIds()
{
    var r = new List<string>();
    foreach (var id in _present.Keys) if (_present[id]
) r.Add(id);
    return r;
}
public int GetFace(string id) => _faces.ContainsKey(id
) ? _faces[id] : -1;
public void SetFace(string id, int f) { if (_faces.Con
tainsKey(id)) _faces[id] = f; }
```

```
    public void ClearSelection()
    {
        if (_selected != "")
        {
            string s = _selected;
            _selected = "";
            OnSelectionChanged?.Invoke(s, false);
        }
    }
    // 消除一对：仅更新数据并通知，视图层负责播放撞击动画与销毁
    public void RemovePair(string a, string b)
    {
        if (_present.ContainsKey(a)) _present[a] = false;
        if (_present.ContainsKey(b)) _present[b] = false;
        OnMatched?.Invoke(a, b);
    }
}
// ===== FILE: Assets\Scripts\Core\LayoutFactory.cs =====
using System.Collections.Generic;
using UnityEngine;
/// <summary>
/// 规则居中塔布局生成：每层比下层缩 1 列/行并居中，形成经典麻将塔。
/// 与现有 mahjong-layout 的“ 缩进塔 ”同构，LevelGenerator.FindPath 可解
    。
/// </summary>
public static class LayoutFactory
{
    public static LayoutData MakeTower(int cols, int rows,
     int layers)
    {
        var tiles = new List<TileDef>();
        int L = Mathf.Max(1, layers);
        for (int l = 0; l < L; l++)
        {
            int c = cols - 2 * l;
            int r = rows - 2 * l;
            if (c <= 0 || r <= 0) break;
            int startX = l * 2;
            int startY = l * 2;
            int idx = 0;
            for (int j = 0; j < r; j++)
                for (int i = 0; i < c; i++)
                {
                    tiles.Add(new TileDef
                    {
                        id = "L" + l + "_T" + (idx++),
                        layer = l,
                        x = startX + i * 2,
                        y = startY + j * 2,
                        width = 2,
                        height = 2
                    });
                }
        }
        return new LayoutData { tiles = tiles.ToArray() };
    }
}
// ===== FILE: Assets\Scripts\Core\LevelGenerator.cs =====
using System;
using System.Collections.Generic;
using System.Text;
using UnityEngine;
/// <summary>
/// 按骨架 JSON 生成有牌布局（Godot lv_fill.gd 的 C# 移植）。
/// 算法：先只依赖几何用 DFS 搜一条「每步两张牌同时自由」的合法消除路径，
/// 再把牌面种类按路径配对序分配下去，天然保证可解。
/// </summary>
```

```
public static class LevelGenerator
{
    const int DEFAULT_SEARCH_BUDGET = 500000;
    const int MAX_ATTEMPTS = 16;
    const int MAX_FACE_TYPES = 36; // 受限于 art/card/ 下的实际牌面
    图数量（icon_001..icon_036）
    private class PathResult
    {
        public bool ok;
        public string error = "";
        public string code = "";
    }
    // 工作态（单线程，每次 Generate 重新初始化）。
    private static System.Random _rng;
    private static List<TileDef> _tiles;
    private static int _n;
    private static List<int>[] _above;
    private static List<int>[] _left;
    private static List<int>[] _right;
    private static bool[] _present;
    private static List<int[]> _path;
    private static int _visited;
    private static int _budget;
    private static bool _budgetHit;
    private static HashSet<string> _failed;
    public static GenResult Generate(LayoutData data, int
    seedValue = -1, int searchBudget = DEFAULT_SEARCH_BUDGET)
    {
        _rng = (seedValue >= 0) ? new System.Random(seedVa
    lue) : new System.Random();
        _tiles = new List<TileDef>(data.tiles);
        var v = Validate(_tiles);
        if (!v.ok) return new GenResult { ok = false, erro
    r = v.error };
        _n = _tiles.Count;
        _budget = searchBudget;
        var geo = BuildGeometry(_tiles);
        _above = geo.above;
        _left = geo.left;
        _right = geo.right;
        GenResult lastErr = null;
        for (int attempt = 1; attempt <= MAX_ATTEMPTS; att
    empt++)
        {
            if (seedValue >= 0) _rng = new System.Random((
    seedValue + attempt * 1000003) & 0x7fffffff);
            else _rng = new System.Random();
            PathResult res = FindPath();
            if (!res.ok)
            {
                lastErr = new GenResult { ok = false, erro
    r = res.error };
                if (res.code == "SEARCH_LIMIT_REACHED") co
    ntinue;
                break;
            }
            int pairCount = _n / 2;
            List<int> typeCounts = ChooseTypeCounts(pairCo
    unt);
            var pairTypes = new List<int>();
            for (int ti = 0; ti < typeCounts.Count; ti++)
                for (int k = 0; k < typeCounts[ti]; k++) p
    airTypes.Add(ti);
            Shuffle(pairTypes);
            var assignments = new Dictionary<string, int>(
    );
            for (int k = 0; k < _path.Count; k++)
```

```
        {
            int face = pairTypes[k];
            int a = _path[k][0];
            int b = _path[k][1];
            assignments[_tiles[a].id] = face;
            assignments[_tiles[b].id] = face;
        }
        PathResult vr = ValidatePath(assignments);
        if (!vr.ok) { lastErr = new GenResult { ok = f
alse, error = vr.error }; continue; }
        var outTiles = new List<TileOut>();
        foreach (var t in _tiles)
            outTiles.Add(new TileOut { id = t.id, laye
r = t.layer, x = t.x, y = t.y, width = t.width, height = t
.height, tileType = assignments[t.id] });
        var faceCounts = new Dictionary<int, int>();
        foreach (var val in assignments.Values)
        {
            if (!faceCounts.ContainsKey(val)) faceCoun
ts[val] = 0;
            faceCounts[val]++;
        }
        return new GenResult
        {
            ok = true,
            attempt = attempt,
            tileCount = _n,
            pairCount = pairCount,
            tiles = outTiles,
            assignments = assignments,
            geometry = GeometryById(),
            faceCounts = faceCounts
        };
    }
    return lastErr ?? new GenResult { ok = false, erro
r = "多次尝试仍未能生成有效牌局" };
}
// ---------- 几何 ----------
private static (List<int>[] above, List<int>[] left, L
ist<int>[] right) BuildGeometry(List<TileDef> tiles)
{
    int n = tiles.Count;
    var above = new List<int>[n];
    var left = new List<int>[n];
    var right = new List<int>[n];
    for (int i = 0; i < n; i++) { above[i] = new List<
int>(); left[i] = new List<int>(); right[i] = new List<int
>(); }
    for (int i = 0; i < n; i++)
    {
        TileDef a = tiles[i];
        for (int j = 0; j < n; j++)
        {
            if (i == j) continue;
            TileDef b = tiles[j];
            if (b.layer > a.layer && Intersects(a, b))
 above[i].Add(j);
            if (b.layer == a.layer && VOverlap(a, b))
            {
                if (b.x + b.width == a.x) left[i].Add(
j);
                if (a.x + a.width == b.x) right[i].Add
(j);
            }
        }
    }
    return (above, left, right);
```

```
    }
    private static bool Intersects(TileDef a, TileDef b)
    {
        return a.x < b.x + b.width && a.x + a.width > b.x
    && a.y < b.y + b.height && a.y + a.height > b.y;
    }
    private static bool VOverlap(TileDef a, TileDef b)
    {
        return a.y < b.y + b.height && a.y + a.height > b.
    y;
    }
    private static bool IsFreeIdx(int i, bool[] arr)
    {
        for (int j = 0; j < _above[i].Count; j++) if (arr[
    _above[i][j]]) return false;
        bool lb = false, rb = false;
        for (int j = 0; j < _left[i].Count; j++) if (arr[_
    left[i][j]]) { lb = true; break; }
        for (int j = 0; j < _right[i].Count; j++) if (arr[
    _right[i][j]]) { rb = true; break; }
        return !(lb && rb);
    }
    // ---------- 搜索一条可解消除路径 ----------
    private static PathResult FindPath()
    {
        _present = new bool[_n];
        for (int i = 0; i < _n; i++) _present[i] = true;
        _path = new List<int[]>();
        _visited = 0;
        _budgetHit = false;
        _failed = new HashSet<string>();
        if (Dfs(_n)) return new PathResult { ok = true };
        if (_budgetHit) return new PathResult { ok = false
    , error = "搜索预算耗尽", code = "SEARCH_LIMIT_REACHED" };
        return new PathResult { ok = false, error = "骨架几何规
    则下不可解", code = "UNSOLVABLE_SKELETON" };
    }
    private static bool Dfs(int remaining)
    {
        if (remaining == 0) return true;
        _visited++;
        if (_visited > _budget) { _budgetHit = true; retur
    n false; }
        string key = StateKey(_present);
        if (_failed.Contains(key)) return false;
        var free = new List<int>();
        for (int i = 0; i < _n; i++) if (_present[i] && Is
    FreeIdx(i, _present)) free.Add(i);
        if (free.Count < 2) { _failed.Add(key); return fal
    se; }
        var ranked = new List<(int i, int urgency, float s
    core)>();
        foreach (var i in free)
        {
            bool lb = false, rb = false;
            for (int j = 0; j < _left[i].Count; j++) if (_
    present[_left[i][j]]) { lb = true; break; }
            for (int j = 0; j < _right[i].Count; j++) if (
    _present[_right[i][j]]) { rb = true; break; }
            int urgency = (lb ? 1 : 0) + (rb ? 1 : 0);
            float score = urgency + ((float)_rng.NextDoubl
    e() - 0.5f) * 0.25f;
            ranked.Add((i, urgency, score));
        }
        ranked.Sort((p, q) => q.score.CompareTo(p.score));
        var pairs = new List<(int i, int j, float reward,
    float tie)>();
```

```
    for (int a = 0; a < ranked.Count; a++)
        for (int b = a + 1; b < ranked.Count; b++)
        {
            float reward = ranked[a].urgency + ranked[
b].urgency;
            float tie = (float)_rng.NextDouble();
            pairs.Add((ranked[a].i, ranked[b].i, rewar
d, tie));
        }
    pairs.Sort((p, q) => q.reward.CompareTo(p.reward)
!= 0 ? q.reward.CompareTo(p.reward) : q.tie.CompareTo(p.ti
e));
    foreach (var p in pairs)
    {
        int i = p.i, j = p.j;
        _present[i] = false;
        _present[j] = false;
        _path.Add(new int[] { i, j });
        if (Dfs(remaining - 2)) return true;
        _path.RemoveAt(_path.Count - 1);
        _present[i] = true;
        _present[j] = true;
        if (_budgetHit) return false;
    }
    _failed.Add(key);
    return false;
}
private static string StateKey(bool[] arr)
{
    var sb = new StringBuilder();
    for (int i = 0; i < arr.Length; i++) sb.Append(arr
[i] ? '1' : '0');
    return sb.ToString();
}
// ---------- 校验 ----------
private static PathResult Validate(List<TileDef> tiles
)
{
    if (tiles.Count == 0) return new PathResult { ok =
 false, error = "tiles 为空" };
    if (tiles.Count % 2 != 0) return new PathResult {
ok = false, error = "牌位数为奇数" };
    var ids = new HashSet<string>();
    foreach (var t in tiles)
    {
        if (string.IsNullOrEmpty(t.id)) return new Pat
hResult { ok = false, error = "牌位缺少 id" };
        if (t.width <= 0 || t.height <= 0) return new
PathResult { ok = false, error = "牌位宽高无效" };
        if (ids.Contains(t.id)) return new PathResult
{ ok = false, error = "牌位 ID 重复: " + t.id };
        ids.Add(t.id);
    }
    for (int i = 0; i < tiles.Count; i++)
        for (int j = i + 1; j < tiles.Count; j++)
        {
            var a = tiles[i];
            var b = tiles[j];
            if (a.layer == b.layer && Intersects(a, b)
)
                return new PathResult { ok = false, er
ror = "同层面积重叠: " + a.id + " 与 " + b.id };
        }
    return new PathResult { ok = true };
}
private static PathResult ValidatePath(Dictionary<stri
ng, int> assignments)
```

```
{
    var present = new bool[_n];
    for (int i = 0; i < _n; i++) present[i] = true;
    for (int step = 0; step < _path.Count; step++)
    {
        int i = _path[step][0];
        int j = _path[step][1];
        if (i == j || !present[i] || !present[j]) retu
rn new PathResult { ok = false, error = "引用无效或已移除的牌位" };
        if (!IsFreeIdx(i, present) || !IsFreeIdx(j, pr
esent)) return new PathResult { ok = false, error = "存在被遮挡
或两侧均封闭的牌" };
        if (assignments[_tiles[i].id] != assignments[_
tiles[j].id]) return new PathResult { ok = false, error =
"牌面不同" };
        present[i] = false;
        present[j] = false;
    }
    for (int i = 0; i < _n; i++) if (present[i]) retur
n new PathResult { ok = false, error = "消除路径结束后仍有牌位未消" };
    return new PathResult { ok = true };
}
private static List<int> ChooseTypeCounts(int pairCoun
t)
{
    int maxKinds = Math.Min(pairCount, MAX_FACE_TYPES)
;
    int kinds = _rng.Next(1, maxKinds + 1); // [1, max
Kinds]
    var counts = new List<int>();
    for (int i = 0; i < kinds; i++) counts.Add(1);
    int extra = pairCount - kinds;
    while (extra > 0) { counts[_rng.Next(kinds)]++; ex
tra--; }
    Shuffle(counts);
    return counts;
}
private static void Shuffle<T>(List<T> arr)
{
    for (int i = arr.Count - 1; i > 0; i--)
    {
        int j = _rng.Next(i + 1);
        T t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }
}
private static Dictionary<string, TileGeo> GeometryByI
d()
{
    var idxToId = new List<string>();
    foreach (var t in _tiles) idxToId.Add(t.id);
    var outGeo = new Dictionary<string, TileGeo>();
    for (int i = 0; i < _n; i++)
    {
        outGeo[idxToId[i]] = new TileGeo
        {
            above = MapIds(_above[i], idxToId),
            left = MapIds(_left[i], idxToId),
            right = MapIds(_right[i], idxToId)
        };
    }
    return outGeo;
}
private static List<string> MapIds(List<int> idxs, Lis
t<string> idxToId)
{
```

```
        var r = new List<string>();
        foreach (var i in idxs) r.Add(idxToId[i]);
        return r;
    }
}
// ===== FILE: Assets\Scripts\Data\GameData.cs =====
using System.Collections.Generic;
// 布局骨架里的一张牌位（来自 mahjong-layout.json）。
[System.Serializable]
public class TileDef
{
    public string id;
    public int layer;
    public int x;
    public int y;
    public int width;
    public int height;
}
[System.Serializable]
public class LayoutData
{
    public TileDef[] tiles;
}
// 来自 ui_config.json。
[System.Serializable]
public class ConfigData
{
    public bool fit_to_screen = true;
    public float padding = 40f;
    public float zoom = 1f;
    public int tile_width = 56;
    public int tile_height = 96;
    public int face_width = 40;
    public int face_height = 64;
    public float shadow_x = 5f;
    public float shadow_y = 7f;
    public float layer_dx = 0f;
    public float layer_dy = 0f;
    public float flip_dur = 0.5f;    // 牌正反面翻转动画总时长（秒）
    // 以下为原 layout_debug.json 合并进来的「配对碰撞/调试」参数
    public float crashSideOffset = 55f;  // 配对后，两张牌在中央点左右
    各偏移的水平距离
    public float liftDur = 0.15f;       // 抬起并移到左右目标点的时长
    public float collideDur = 0.14f;    // 两牌从左右目标点相撞到中央
    点的时长
    public float shrinkScale = 0.3f;    // 碰撞瞬间缩放到的目标比例
    public float shrinkDur = 0.14f;     // 缩小动画时长
    public float flyDur = 0.45f;        // 缩小后飞向图鉴按钮的时长
    public float pauseDur = 0.5f;       // 移到左右目标点后、相撞前的
    停顿时长
    public bool show_offset_sliders = false; // 调试：是否显示 X/
    Y 层偏移滑条
}
// 生成后的一张牌（带牌面种类）。
[System.Serializable]
public class TileOut
{
    public string id;
    public int layer;
    public int x;
    public int y;
    public int width;
    public int height;
    public int tileType;
}
// 交互层判定「自由牌」所需的几何关系（id 列表）。
public class TileGeo
```

```
{
    public List<string> above = new List<string>();
    public List<string> left = new List<string>();
    public List<string> right = new List<string>();
}
// 关卡生成结果（仅在内存中使用，无需可序列化）。
public class GenResult
{
    public bool ok;
    public string error;
    public int attempt;
    public int tileCount;
    public int pairCount;
    public List<TileOut> tiles;
    public Dictionary<string, int> assignments;
    public Dictionary<string, TileGeo> geometry;
    public Dictionary<int, int> faceCounts;
}
// ===== FILE: Assets\Scripts\Data\LevelData.cs =====
using System;
// 单关卡文件（Resources/level/levelN.json）：
// - 若提供 tiles 则直接作为布局（多布局文件，真实几何）；
// - 否则用 cols/rows/layers 由 LayoutFactory 生成占位塔布局。
[Serializable]
public class LevelFile
{
    public string name;      // 显示名（空则用 "第N关"）
    public int targetScore;  // 通关目标分（展示用）
    public int seed;         // 牌面分配随机种子（0 亦可，确定性）
    public int cols;         // 生成塔底层列数（tiles 为空时生效）
    public int rows;         // 生成塔底层行数
    public int layers;       // 生成塔层数
    public TileDef[] tiles;  // 可选：直接布局
}
// ===== FILE: Assets\Scripts\Tests\HeadlessTest.cs =====
using System;
using System.Collections.Generic;
using System.Reflection;
using UnityEngine;
/// <summary>
/// 无头（headless）冒烟测试：在 Unity 批处理模式下验证逻辑层与资源加载，
/// 不依赖渲染。运行：Unity -batchmode -executeMethod HeadlessTest.
    Run
/// </summary>
public static class HeadlessTest
{
    public static void Run()
    {
        Debug.Log("[HT] === start ===");
        // 1) 资源加载：Card/ 下是多精灵图集，必须用 LoadAll<Sprite>
        var cf = Resources.LoadAll<Sprite>("Card/card_fron
    t");
        Debug.Log("[HT] card_front sub-sprites=" + (cf !=
    null ? cf.Length : 0));
        bool miss = false;
        for (int i = 1; i <= 36; i++)
        {
            var s = Resources.LoadAll<Sprite>("Card/icon_"
     + i.ToString("D3"));
            if (s == null || s.Length == 0) { Debug.LogErr
    or("[HT] MISSING icon_" + i); miss = true; }
        }
        Debug.Log("[HT] face sprites " + (miss ? "HAS MISS
    ING" : "all 36 OK"));
        // 2) 布局解析 + 关卡生成
        var layout = Resources.Load<TextAsset>("mahjong-la
    yout");
```

```
    if (layout == null) { Debug.LogError("[HT] MISSING
 mahjong-layout"); return; }
    var data = JsonUtility.FromJson<LayoutData>(layout
.text);
    Debug.Log("[HT] layout tiles parsed=" + (data.tile
s != null ? data.tiles.Length : 0));
    var gen = LevelGenerator.Generate(data);
    if (!gen.ok) { Debug.LogError("[HT] GENERATE FAILE
D: " + gen.error); return; }
    Debug.Log("[HT] generated tileCount=" + gen.tileCo
unt + " pairs=" + gen.pairCount + " attempts=" + gen.attem
pt);
    // 3) 模拟消除：验证规则与可解性（贪心：每步取任意一对同面且都自由的牌）
    var board = new GameBoard();
    board.Setup(gen.geometry, gen.assignments);
    int steps = 0;
    while (!board.IsWin())
    {
        var freeByFace = new Dictionary<int, string>()
;
        string a = null, b = null;
        foreach (var id in board.PresentIds())
        {
            if (!board.IsFree(id)) continue;
            int f = board.GetFace(id);
            if (freeByFace.ContainsKey(f)) { a = freeB
yFace[f]; b = id; break; }
            freeByFace[f] = id;
        }
        if (a == null || b == null)
        {
            Debug.LogError("[HT] NO VALID MOVE at step
 " + steps + " remaining=" + board.Remaining() + " (贪心死局不一
定是关卡不可解)");
            break;
        }
        board.Click(a);
        board.Click(b);
        steps++;
        if (steps > 100000) { Debug.LogError("[HT] too
 many steps"); break; }
    }
    if (board.IsWin()) Debug.Log("[HT] WIN in " + step
s + " steps - 关卡可解/规则正常");
    else Debug.LogError("[HT] 未清空 remaining=" + board.
Remaining());
    // 4) 死局检测逻辑健康性：构造一个明显死局（两张不同面且互相封锁）应触发 OnDeadlock
 而非异常
    var dead = new GameBoard();
    var g = new Dictionary<string, TileGeo>();
    g["x1"] = new TileGeo(); g["x2"] = new TileGeo();
    g["x1"].right.Add("x2"); g["x2"].left.Add("x1");
    var fc = new Dictionary<string, int> { { "x1", 1 }
, { "x2", 2 } };
    dead.Setup(g, fc);
    Debug.Log("[HT] deadlock-has-valid-move=" + dead.H
asValidMove() + " (应为 False)");
    // 5) 模拟 GameController 启动：验证 BuildUI/LoadResource
s/BuildBoard 不抛异常（覆盖 UI 构建层）
    try
    {
        var go = new GameObject("TestGC");
        var gc = go.AddComponent<GameController>();
        var start = typeof(GameController).GetMethod("
Start", BindingFlags.NonPublic | BindingFlags.Instance);
        start.Invoke(gc, null);
        Debug.Log("[HT] GameController.Start 执行无异常（Bui
```

```
  ldUI/LoadResources/BuildBoard 通过）");
        UnityEngine.Object.DestroyImmediate(go);
    }
    catch (System.Exception e)
    {
      Debug.LogError("[HT] GameController.Start thre
  w: " + e);
    }
    Debug.Log("[HT] === done ===");
  }
}
// ===== FILE: Assets\Scripts\UI\CodexItem.cs =====
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// 图鉴行项预制体组件（挂在 CodexItem 预制体根）。
/// 预制体结构：根（HorizontalLayoutGroup） + Icon(Image) + Txt(Text
  ) + Color(Image)。
/// Bind 设置花色图标 / 数量 / 等级色块或等级背图。
/// </summary>
public class CodexItem : MonoBehaviour
{
  private Image _icon;
  private Text _txt;
  private Image _color;
  private void Awake()
  {
    _icon = transform.Find("Icon")?.GetComponent<Image
  >();
    _txt = transform.Find("Txt")?.GetComponent<Text>()
  ;
    _color = transform.Find("Color")?.GetComponent<Ima
  ge>();
  }
  public void Bind(Sprite face, int count, int lvl, Spri
  te levelBack, Color codeColor)
  {
    if (_icon != null) { _icon.sprite = face; _icon.pr
  eserveAspect = true; }
    if (_txt != null) _txt.text = "x" + count + "  Lv"
   + lvl;
    if (_color != null)
    {
      _color.preserveAspect = true;
      if (levelBack != null) _color.sprite = levelBa
  ck;
      else _color.color = codeColor;
    }
  }
}
// ===== FILE: Assets\Scripts\UI\CodexPanel.cs =====
using System;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
using System.Collections.Generic;
/// <summary>
/// 图鉴面板组件（挂在 CodexPanel 预制体根）。预制体含：标题 + CodexScroll(Scrol
  lRect>Viewport>Content) + CloseBtn。
/// Init 绑定关闭回调；Refresh 按数据运行时实例化 CodexItem 预制体并 Bind。
/// </summary>
public class CodexPanel : MonoBehaviour
{
  private Transform _content;
  public void Init(UnityAction onClose)
  {
    _content = FindDeep(transform, "Content");
```

```
    var closeBtn = FindDeep(transform, "CloseBtn")?.Ge
tComponent<Button>();
    if (closeBtn == null) closeBtn = FindButtonByText(
transform, "关闭");
    closeBtn?.onClick.AddListener(onClose);
}
public void Refresh(List<Sprite> faces, int[] collecte
d, Sprite[] levelBacks, Color[] codeColors)
{
    if (_content == null) return;
    var children = new List<GameObject>();
    for (int i = 0; i < _content.childCount; i++) chil
dren.Add(_content.GetChild(i).gameObject);
    foreach (var c in children) UnityEngine.Object.Des
troy(c);
    var prefab = Resources.Load<GameObject>("Prefab/Co
dexItem");
    var font = UiTools.LoadFont();
    for (int f = 0; f < faces.Count; f++)
    {
        int count = (f < collected.Length) ? collected
[f] : 0;
        int lvl = LevelOf(count);
        if (prefab != null)
        {
            var go = Instantiate(prefab, _content);
            var item = go.GetComponent<CodexItem>();
            if (item != null)
            {
                Sprite lb = (lvl >= 0 && lvl < levelBa
cks.Length) ? levelBacks[lvl] : null;
                Color cc = (codeColors != null && lvl
>= 0 && lvl < codeColors.Length) ? codeColors[lvl] : Color
.gray;
                item.Bind(faces[f], count, lvl, lb, cc
);
            }
        }
        else
        {
            // 回退：CodexItem 预制体缺失时运行时构建行
            BuildRowFallback(_content, f, faces[f], co
unt, lvl, levelBacks, codeColors, font);
        }
    }
}
// 回退：CodexItem 预制体缺失时运行时构建行（逻辑同原 RefreshCodex）
private static void BuildRowFallback(Transform parent,
 int f, Sprite face, int count, int lvl, Sprite[] levelBac
ks, Color[] codeColors, Font font)
{
    var row = UiTools.MakeGO("Row" + f, parent);
    var rtr = row.GetComponent<RectTransform>(); rtr.s
izeDelta = new Vector2(0, 64);
    var hlg = row.AddComponent<HorizontalLayoutGroup>(
);
    hlg.spacing = 12; hlg.childAlignment = TextAnchor.
MiddleLeft;
    hlg.childForceExpandWidth = false; hlg.childForceE
xpandHeight = false;
    var icon = UiTools.MakeGO("Icon", row.transform);
    var itr = icon.GetComponent<RectTransform>(); itr.
sizeDelta = new Vector2(48, 48);
    var iimg = icon.AddComponent<Image>(); iimg.preser
veAspect = true; iimg.sprite = face;
    var txt = UiTools.MakeGO("Txt", row.transform);
    var t2 = txt.AddComponent<Text>();
```

```
    t2.text = "x" + count + "  Lv" + lvl; t2.fontSize
  = 24; t2.alignment = TextAnchor.MiddleLeft;
    if (font != null) t2.font = font;
    var ttr2 = txt.GetComponent<RectTransform>(); ttr2
  .sizeDelta = new Vector2(200, 48);
    var col = UiTools.MakeGO("Color", row.transform);
    var c2 = col.GetComponent<RectTransform>(); c2.siz
  eDelta = new Vector2(48, 48);
    var cimg = col.AddComponent<Image>(); cimg.preserv
  eAspect = true;
    if (lvl >= 0 && lvl < levelBacks.Length && levelBa
  cks[lvl] != null) cimg.sprite = levelBacks[lvl];
    else if (codeColors != null && lvl >= 0 && lvl < c
  odeColors.Length) cimg.color = codeColors[lvl];
    else cimg.color = Color.gray;
  }
  private static Transform FindDeep(Transform root, stri
  ng name)
  {
    foreach (Transform c in root)
    {
      if (c.name == name) return c;
      var d = FindDeep(c, name);
      if (d != null) return d;
    }
    return null;
  }
  // 兜底：递归找文本为指定内容的 Button（防止预制体按钮改名后找不到）
  private static Button FindButtonByText(Transform root,
   string text)
  {
    var btn = root.GetComponent<Button>();
    if (btn != null)
    {
      var t = root.GetComponentInChildren<Text>();
      if (t != null && t.text == text) return btn;
    }
    foreach (Transform c in root)
    {
      var b = FindButtonByText(c, text);
      if (b != null) return b;
    }
    return null;
  }
  private static int LevelOf(int count) => count >= 1000
   ? 3 : (count >= 100 ? 2 : (count >= 10 ? 1 : 0));
}
// ===== FILE: Assets\Scripts\UI\GameController.cs =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.InputSystem.UI;
using UnityEngine.UI;
/// <summary>
/// 完整游戏主控制器（Godot game_session.gd 的 C# 移植）。
/// 无限随机模式：随机生成可解牌阵，全消即过关并自动进下一局，分数跨局累加。
/// 不限时；提示（限次）高亮一对可消牌；死局弹提示，玩家可洗牌或重开；也可手动洗牌。
/// 最高分持久化。所有 UI 在运行时构建，场景文件无需引用本脚本。
/// </summary>
public class GameController : MonoBehaviour
{
  const float COMBO_SEC = 3.0f;
  const int HINTS_PER_ROUND = 5;
  const int ROUND_BONUS = 500;
  const int BASE_SCORE = 100;
  private GameBoard _board;
```

```
private Dictionary<string, TileView> _tiles = new Dict
ionary<string, TileView>();
private GameObject _tilePrefab;
private Sprite _cardFront;
private Sprite[] _levelBacks = new Sprite[4];  // 等级0
~3 对应的背面图（Resources/CardPanel/Level0..3）
private Dictionary<int, Sprite> _faceSprites = new Dic
tionary<int, Sprite>();
private LayoutData _layoutData;
private ConfigData _cfg = new ConfigData();
private int _score;
private int _best;
private int _round;
private int _combo;
private float _lastMatchTime;
private int _hints;
private bool _busy;
private Text _scoreLabel;
private Text _bestLabel;
private Text _roundLabel;
private Text _toast;
private GameObject _toastGO;
private Button _hintBtn;
private Button _shuffleBtn;
private Transform _uiRoot;
private GameObject _deadlockOverlay;
private GameState _codexReturnState = GameState.Playin
g;
private GameObject _fade;
private CanvasGroup _fadeCg;
private bool _transitioning;
private Button _restartBtn;
private RectTransform _boardRoot;
private Font _font;
private float _layerDx;
private float _layerDy;
private Slider _xOffsetSlider; // 调试：X 轴层偏移滑条
private Slider _yOffsetSlider; // 调试：Y 轴层偏移滑条
private const float X_OFFSET_MIN = -40f, X_OFFSET_MAX
= 40f;
private const float Y_OFFSET_MIN = -40f, Y_OFFSET_MAX
= 40f;
private string _firstPick = "";              // 当前选
中的第一张牌 id
private Vector2 _asideOffset = new Vector2(40, 40); //
 保留（兼容）
private List<(TileOut t, GameObject go)> _placedView =
 new List<(TileOut t, GameObject go)>();
// 图鉴 / 对象池 / 音频
private int[] _codexCollected = new int[0];    // 每种
花色收集数（按 faceIndex）
private Button _codexBtn;
private GameObject _codexPanel;
private AudioSource _audio;
private AudioClip _peng;
private AudioClip _bgm;
private AudioClip _fail;
private AudioClip _select0;
private AudioClip _select1;
private AudioSource _bgmSrc;   // 背景音乐专用音源（与音效分开，便于独
立开关）
private SaveManager.SettingsData _settings; // 设置数据（音
乐/音效/震动/画质/语言）
private GameObject _settingsPanel;
private float _flipDur = 0.5f;  // 正反面翻转动画总时长（来自配置 fl
ip_dur）
private List<TileView> _pool = new List<TileView>(); /
```

```
/ 牌对象池
private static readonly Color[] CODE_COLORS = {
    new Color(0.95f, 0.95f, 0.95f), // Lv0 白
    new Color(0.40f, 0.85f, 0.40f), // Lv1 绿
    new Color(0.40f, 0.60f, 1.00f), // Lv2 蓝
    new Color(1.00f, 0.80f, 0.20f), // Lv3 金
};
// 关卡 / 状态机
private enum GameState { Menu, LevelSelect, Playing, P
aused, Result }
private GameState _state = GameState.Menu;
private LevelEntry[] _levels = new LevelEntry[0];
private int _currentLevel = 0;
private GameObject _topBar, _bottomBar;
private GameObject _menuPanel, _levelPanel, _pausePane
l, _resultPanel;
private Button _pauseBtn;
// 面板组件引用（加载预制体后绑定）
private MenuPanel _menuPanelC;
private LevelSelectPanel _levelPanelC;
private PausePanel _pausePanelC;
private ResultPanel _resultPanelC;
private CodexPanel _codexPanelC;
private float _nativeW;
private float _nativeH;
private int _lastScreenW;
private int _lastScreenH;
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadTy
pe.AfterSceneLoad)]
private static void Boot()
{
    if (Object.FindFirstObjectByType<GameController>()
 != null) return;
    var go = new GameObject("GameController");
    go.AddComponent<GameController>();
}
// Unity 6 废弃了内置字体名 "Arial.ttf"，改用 "LegacyRuntime.ttf"
；旧版本回退 Arial；
// 两者都失败则返回 null（Text 会使用引擎默认字体，不崩溃）。
private static Font LoadBuiltinFont()
{
    try { return Resources.GetBuiltinResource<Font>("L
egacyRuntime.ttf"); } catch { }
    try { return Resources.GetBuiltinResource<Font>("A
rial.ttf"); } catch { }
    return null;
}
private void Start()
{
    _font = LoadBuiltinFont();
    LoadResources(); // 先于 BuildUI：让 _cfg 在 BuildUI 建滑
条时可用
    _settings = SaveManager.LoadSettings(); // 提前加载：Bu
ildUI 建设置面板时需读取开关状态
    BuildUI();
    _best = SaveManager.LoadBest();
    _codexCollected = SaveManager.LoadCodex();
    EnsureCodexSize();
    _audio = gameObject.AddComponent<AudioSource>();
// 音效音源
    _bgmSrc = gameObject.AddComponent<AudioSource>();
// 背景音乐音源（独立开关）
    ApplyAudioSettings();
    ApplyQualitySetting();
    ShowMenu();
}
private void Update()
```

```
{
    if (_state == GameState.Playing && (Screen.width !
= _lastScreenW || Screen.height != _lastScreenH))
        FitBoard();
}
// ---------- 资源加载 ----------
// 资源均按 multi-sprite 图集导入，Resources.Load<Sprite> 对多精灵资
产会返回 null，
// 必须用 LoadAll<Sprite> 取子精灵（每张图仅一个子精灵，取 [0]）。
private Sprite LoadFirstSprite(string path)
{
    var all = Resources.LoadAll<Sprite>(path);
    return (all != null && all.Length > 0) ? all[0] :
null;
}
private void LoadResources()
{
    _cardFront = LoadFirstSprite("Card/card_front");
    _faceSprites.Clear();
    for (int i = 1; i <= 36; i++)
    {
        _faceSprites[i - 1] = LoadFirstSprite("Card/ic
on_" + i.ToString("D3"));
    }
    LoadLevels();
    // 等级背面图：Resources/CardPanel/Level0..3.png（不同等级花色用
不同背面）
    for (int i = 0; i < _levelBacks.Length; i++)
    {
        var s = Resources.Load<Sprite>("CardPanel/Leve
l" + i);
        if (s != null) _levelBacks[i] = s;
    }
    var cfgAsset = Resources.Load<TextAsset>("ui_confi
g");
    if (cfgAsset != null)
    {
        try
        {
            string txt = cfgAsset.text;
            // Unity JsonUtility 对 UTF-8 BOM（\uFEFF）敏感
，开头有它即报 Invalid value，先剥离
            if (!string.IsNullOrEmpty(txt) && txt[0] =
= '\uFEFF') txt = txt.Substring(1);
            _cfg = JsonUtility.FromJson<ConfigData>(tx
t);
        }
        catch (System.Exception e)
        {
            // Unity JsonUtility 在 JSON 语法错误（如文件含 UTF-
8 BOM、// 注释、尾随逗号、true/TRUE 大小写）时会抛异常
            Debug.LogError("[CFG] ui_config.json 解析失败：
" + e.Message + "（请检查是否含 BOM/注释/尾随逗号，已退回默认配置）");
            _cfg = new ConfigData();
        }
        if (_cfg == null) Debug.LogWarning("[CFG] ui_c
onfig.json 解析失败（检查 JSON 语法，如尾随逗号/缺引号），已退回默认配置");
    }
    else Debug.LogWarning("[CFG] 未找到 Resources/ui_conf
ig，已退回默认配置");
    _peng = Resources.Load<AudioClip>("audio/effect/pe
ng"); // 修正路径：资源在 Resources/audio/effect/peng.ogg
    _bgm = Resources.Load<AudioClip>("audio/bgm/bgm_0"
);    // 背景音乐：Resources/audio/bgm/bgm_0.ogg
    _fail = Resources.Load<AudioClip>("audio/effect/fa
il");    // 点击无效牌
    _select0 = Resources.Load<AudioClip>("audio/effect
```

```
/select0"); // 选中有效牌
    _select1 = Resources.Load<AudioClip>("audio/effect
/select1"); // 配对成功的第二张
    _flipDur = (_cfg != null) ? _cfg.flip_dur : 0.5f;
// 翻转用时（配置表 flip_dur）
    _tilePrefab = Resources.Load<GameObject>("Prefab/T
ilePrefab");
}
private Sprite GetFaceSprite(int idx) => (_faceSprites
.ContainsKey(idx) ? _faceSprites[idx] : null);
// ---------- UI 构建 ----------
private GameObject MakeGO(string name, Transform paren
t)
{
    // 所有 MakeGO 创建的节点最终都作为 Canvas/RectTransform 的子节点并
需要直接
    // 操作 RectTransform（锚点/尺寸/偏移），因此创建时即附带 RectTransfo
rm，
    // 否则 TopBar/BottomBar/BoardRoot/Toast/tileGO 等容器节
点 GetComponent 会返回 null 崩溃。
    var go = new GameObject(name, typeof(RectTransform
));
    go.transform.SetParent(parent, false);
    return go;
}
private Text AddText(GameObject parent, string txt, in
t fontSize)
{
    var go = MakeGO("Text", parent.transform);
    var t = go.AddComponent<Text>();
    t.text = txt;
    t.fontSize = fontSize;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = Color.white;
    if (_font != null) t.font = _font;
    var tr = go.GetComponent<RectTransform>();
    tr.anchorMin = Vector2.zero;
    tr.anchorMax = Vector2.one;
    tr.offsetMin = Vector2.zero;
    tr.offsetMax = Vector2.zero;
    return t;
}
private Button AddButton(GameObject parent, string lab
el)
{
    var go = MakeGO("Button", parent.transform);
    var img = go.AddComponent<Image>();
    img.color = new Color(0.20f, 0.50f, 0.82f);
    var btn = go.AddComponent<Button>();
    btn.targetGraphic = img;
    var txt = MakeGO("Text", go.transform);
    var t = txt.AddComponent<Text>();
    t.text = label;
    t.fontSize = 26;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = Color.white;
    if (_font != null) t.font = _font;
    var tr = txt.GetComponent<RectTransform>();
    tr.anchorMin = Vector2.zero;
    tr.anchorMax = Vector2.one;
    tr.offsetMin = Vector2.zero;
    tr.offsetMax = Vector2.zero;
    return btn;
}
private void BuildUI()
{
    // Canvas
```

```
    Canvas canvas = Object.FindFirstObjectByType<Canva
s>();
    GameObject canvasGO;
    if (canvas == null)
    {
        canvasGO = new GameObject("GameCanvas");
        canvas = canvasGO.AddComponent<Canvas>();
        canvas.renderMode = RenderMode.ScreenSpaceOver
lay;
        canvas.sortingOrder = 10;
        canvasGO.AddComponent<CanvasScaler>().uiScaleM
ode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
        var scaler = canvasGO.GetComponent<CanvasScale
r>();
        scaler.referenceResolution = new Vector2(1080,
 1920);
        scaler.matchWidthOrHeight = 0.5f;
        canvasGO.AddComponent<GraphicRaycaster>();
    }
    else
    {
        canvasGO = canvas.gameObject;
    }
    Transform canvasTr = canvasGO.transform;
    // EventSystem（新输入系统）
    if (Object.FindFirstObjectByType<EventSystem>() ==
 null)
    {
        var es = new GameObject("EventSystem");
        es.AddComponent<EventSystem>();
        es.AddComponent<InputSystemUIInputModule>();
    }
    if (!LoadGamePrefab(canvasTr))
    {
    // 顶栏：分数 / 最高分 / 局数
    var top = MakeGO("TopBar", canvasTr);
    _topBar = top;
    var ttr = top.GetComponent<RectTransform>();
    ttr.anchorMin = new Vector2(0, 1);
    ttr.anchorMax = new Vector2(1, 1);
    ttr.pivot = new Vector2(0.5f, 1);
    ttr.sizeDelta = new Vector2(0, 80);
    ttr.anchoredPosition = new Vector2(0, -40);
    var topLg = top.AddComponent<HorizontalLayoutGroup
>();
    topLg.padding = new RectOffset(16, 16, 8, 8);
    topLg.spacing = 24;
    topLg.childAlignment = TextAnchor.MiddleCenter;
    _scoreLabel = AddText(top, "分数 0", 28);
    _bestLabel = AddText(top, "最高 0", 28);
    _roundLabel = AddText(top, "第 1 局", 28);
    foreach (var t in new[] { _scoreLabel, _bestLabel,
 _roundLabel })
    {
        var le = t.gameObject.AddComponent<LayoutEleme
nt>();
        le.flexibleWidth = 1;
    }
    // 底栏：提示 / 洗牌 / 重开
    var bottom = MakeGO("BottomBar", canvasTr);
    _bottomBar = bottom;
    var btr = bottom.GetComponent<RectTransform>();
    btr.anchorMin = new Vector2(0, 0);
    btr.anchorMax = new Vector2(1, 0);
    btr.pivot = new Vector2(0.5f, 0);
    btr.sizeDelta = new Vector2(0, 96);
    btr.anchoredPosition = new Vector2(0, 48);
```

```
    var botLg = bottom.AddComponent<HorizontalLayoutGr
oup>();
    botLg.padding = new RectOffset(16, 16, 8, 8);
    botLg.spacing = 16;
    botLg.childAlignment = TextAnchor.MiddleCenter;
    botLg.childForceExpandWidth = true;
    _hintBtn = AddButton(bottom, "提示(5)");
    _shuffleBtn = AddButton(bottom, "洗牌");
    _restartBtn = AddButton(bottom, "重开");
    foreach (var b in new[] { _hintBtn, _shuffleBtn, _
restartBtn })
    {
        var le = b.gameObject.AddComponent<LayoutEleme
nt>();
        le.flexibleWidth = 1;
        le.minHeight = 64;
    }
    _hintBtn.onClick.AddListener(OnHint);
    _shuffleBtn.onClick.AddListener(OnShuffle);
    _restartBtn.onClick.AddListener(OnRestart);
    // X 轴层偏移滑条（调试用：实时重排，观察 2.5D 堆叠水平错位）
    var xLabel = AddText(bottom, "X层偏移", 22);
    float x0 = (_cfg != null) ? _cfg.layer_dx : 0f;
    _xOffsetSlider = AddSlider(bottom.transform, "XOff
setSlider", X_OFFSET_MIN, X_OFFSET_MAX, x0, v =>
    {
        _layerDx = v;
        LayoutTiles();
    });
    var xle = _xOffsetSlider.gameObject.AddComponent<L
ayoutElement>();
    xle.minHeight = 36; xle.preferredWidth = 240; xle.
flexibleWidth = 1;
    // Y 轴层偏移滑条（调试用：实时重排，观察 2.5D 堆叠高度）
    var yLabel = AddText(bottom, "Y层偏移", 22);
    float y0 = (_cfg != null) ? _cfg.layer_dy : 0f;
    _yOffsetSlider = AddSlider(bottom.transform, "YOff
setSlider", Y_OFFSET_MIN, Y_OFFSET_MAX, y0, v =>
    {
        _layerDy = v;
        LayoutTiles();
    });
    var yle = _yOffsetSlider.gameObject.AddComponent<L
ayoutElement>();
    yle.minHeight = 36; yle.preferredWidth = 240; yle.
flexibleWidth = 1;
    // 调试滑条显隐：由配置 show_offset_sliders 统一控制（默认 false 不显
示）
    bool showSliders = (_cfg != null) ? _cfg.show_offs
et_sliders : false;
    if (_xOffsetSlider != null) _xOffsetSlider.gameObj
ect.SetActive(showSliders);
    if (xLabel != null) xLabel.gameObject.SetActive(sh
owSliders);
    if (_yOffsetSlider != null) _yOffsetSlider.gameObj
ect.SetActive(showSliders);
    if (yLabel != null) yLabel.gameObject.SetActive(sh
owSliders);
    // 牌盘容器
    var br = MakeGO("BoardRoot", canvasTr);
    var brtr = br.GetComponent<RectTransform>();
    brtr.anchorMin = brtr.anchorMax = new Vector2(0.5f
, 0.5f);
    brtr.pivot = new Vector2(0.5f, 0.5f);
    _boardRoot = br.GetComponent<RectTransform>();
    // 居中提示文字
    _toastGO = MakeGO("Toast", canvasTr);
```

```
    var tts = _toastGO.GetComponent<RectTransform>();
    tts.anchorMin = tts.anchorMax = new Vector2(0.5f,
0.5f);
    tts.pivot = new Vector2(0.5f, 0.5f);
    tts.sizeDelta = new Vector2(420, 90);
    _toast = AddText(_toastGO, "", 34);
    _toastGO.SetActive(false);
    // 游戏中暂停按钮（顶栏右侧）
    _pauseBtn = AddButton(top, "暂停");
    var pLe = _pauseBtn.gameObject.AddComponent<Layout
Element>();
    pLe.minHeight = 64; pLe.flexibleWidth = 1;
    _pauseBtn.onClick.AddListener(OnPause);
    _codexBtn = AddButton(top, "图鉴");
    var cLe = _codexBtn.gameObject.AddComponent<Layout
Element>();
    cLe.minHeight = 64; cLe.flexibleWidth = 1;
    _codexBtn.onClick.AddListener(ShowCodex);
    }
    BuildPanels(canvasTr);
    SetGameUiVisible(false);
    }
// 加载游戏界面（HUD）预制体；成功返回 true，缺失/无组件时返回 false（调用方回退到运行时
MakeGO）
private bool LoadGamePrefab(Transform parent)
{
    var prefab = Resources.Load<GameObject>("Prefab/Ga
mePanel");
    if (prefab == null) return false;
    var go = Instantiate(prefab, parent);
    var gp = go.GetComponent<GamePanel>();
    if (gp == null) { Destroy(go); return false; }
    _topBar = gp.topBar != null ? gp.topBar.gameObject
 : null;
    _bottomBar = gp.bottomBar != null ? gp.bottomBar.g
ameObject : null;
    _boardRoot = gp.boardRoot;
    _toastGO = gp.toast != null ? gp.toast.gameObject
: null;
    _toast = gp.toastText;
    _scoreLabel = gp.scoreLabel; _bestLabel = gp.bestL
abel; _roundLabel = gp.roundLabel;
    _hintBtn = gp.hintBtn; _shuffleBtn = gp.shuffleBtn
; _restartBtn = gp.restartBtn;
    _pauseBtn = gp.pauseBtn; _codexBtn = gp.codexBtn;
    _xOffsetSlider = gp.xOffsetSlider; _yOffsetSlider
= gp.yOffsetSlider;
    float x0 = (_cfg != null) ? _cfg.layer_dx : 0f;
    float y0 = (_cfg != null) ? _cfg.layer_dy : 0f;
    bool showSliders = (_cfg != null) ? _cfg.show_offs
et_sliders : false;
    gp.Init(OnHint, OnShuffle, OnRestart, OnPause, Sho
wCodex,
        X_OFFSET_MIN, X_OFFSET_MAX, x0, v => { _la
yerDx = v; LayoutTiles(); },
        Y_OFFSET_MIN, Y_OFFSET_MAX, y0, v => { _la
yerDy = v; LayoutTiles(); },
        showSliders);
    return true;
}
// ---------- 局流程 ----------
private void StartRound()
{
    PlayBgm(); // 游戏开始播放背景音乐
    _busy = false; // 新一局开始，清除通关/结算时的忙碌锁，恢复输入
    _round = 1;
    _hints = HINTS_PER_ROUND;
```

```
    _combo = 0;
    BuildBoard();
    UpdateHud();
    FitBoard();
    ShowToast(_levels != null && _currentLevel >= 0 &&
 _currentLevel < _levels.Length ? _levels[_currentLevel].n
ame : "第 1 局");
}
private void BuildBoard()
{
    if (_layoutData == null || _layoutData.tiles == nu
ll || _layoutData.tiles.Length == 0)
    {
        Debug.LogError("GameController: 布局数据为空，无法生成牌阵"
);
        return;
    }
    // 回收所有现有牌到对象池（不销毁实例）
    foreach (var kv in _tiles) ReleaseTile(kv.Value);
    _tiles.Clear();
    _placedView.Clear();
    // 2.5D 层偏移取自配置（默认 x=0 纯垂直、y=5.5 每层上抬），不再运行时可调
    _layerDx = (_cfg != null) ? _cfg.layer_dx : 0f;
    _layerDy = (_cfg != null) ? _cfg.layer_dy : 0f;
    _firstPick = "";
    _asideOffset = (_cfg != null) ? new Vector2(_cfg.t
ile_width * 0.8f, _cfg.tile_height * 0.8f) : new Vector2(4
0f, 40f);
    if (_xOffsetSlider != null) _xOffsetSlider.value =
 _layerDx; // 重开本关时滑条回到配置默认值
    if (_yOffsetSlider != null) _yOffsetSlider.value =
 _layerDy; // 重开本关时滑条回到配置默认值
    float tw = _cfg.tile_width;
    float th = _cfg.tile_height;
    int seed = (_levels != null && _currentLevel >= 0
&& _currentLevel < _levels.Length) ? _levels[_currentLevel
].seed : Random.Range(0, int.MaxValue);
    var gen = LevelGenerator.Generate(_layoutData, see
d);
    if (!gen.ok)
    {
        Debug.LogError("GameController: 生成失败 -> " + ge
n.error);
        return;
    }
    _board = new GameBoard();
    _board.Setup(gen.geometry, gen.assignments);
    _board.OnSelectionChanged += OnSelectionChanged;
    _board.OnMatched += OnMatched;
    _board.OnBlocked += OnBlocked;
    _board.OnWin += OnWin;
    _board.OnDeadlock += OnDeadlock;
    // 仅创建牌（含预制体 Shadow/Body/Face 与 TileView），定位交给 Lay
outTiles
    foreach (var t in gen.tiles)
    {
        TileView tv = GetTile();
        tv.tileId = t.id;
        tv.faceIndex = t.tileType;
        tv.ResetView();
        tv.transform.SetParent(_boardRoot, false);
        var tr = tv.GetComponent<RectTransform>();
        tr.anchorMin = tr.anchorMax = new Vector2(0.5f
, 0.5f);
        tr.pivot = new Vector2(0.5f, 0.5f);
        tr.sizeDelta = new Vector2(tw, th);
        // 阴影层：完全按预制体生成（尺寸/锚点/偏移/颜色都由你微调），代码只回填缺失的牌底图
```

```
        Image shadowImg = FindDeep(tv.transform, "Shad
ow")?.GetComponent<Image>();
        if (shadowImg == null)
        {
            shadowImg = MakeGO("Shadow", tv.transform)
.AddComponent<Image>();
            shadowImg.color = new Color(0f, 0f, 0f, 0.
35f);
            shadowImg.preserveAspect = true;
        }
        if (shadowImg.sprite == null) shadowImg.sprite
 = _cardFront;
        shadowImg.raycastTarget = false;
        // 牌内绘制顺序（Shadow 在最下）由预制体 RootOrder 决定，代码不再强制
        // 牌底：完全由预制体提供，代码只引用、不新建也不覆盖其图片（去掉 bodyImg 的新增
逻辑）
        Image bodyImg = FindDeep(tv.transform, "Body (
2)")?.GetComponent<Image>();
        if (bodyImg == null) bodyImg = FindDeep(tv.tra
nsform, "Body")?.GetComponent<Image>();
        if (bodyImg == null) bodyImg = FindDeep(tv.tra
nsform, "Body_1")?.GetComponent<Image>(); // 回退到 Body_1 层
        if (bodyImg != null) bodyImg.raycastTarget = t
rue;
        // 牌面：牌面 sprite 必须按 tileType 绑定（每张不同），其余完全按预制体
        Image faceImg = FindDeep(tv.transform, "Face")
?.GetComponent<Image>();
        if (faceImg == null)
        {
            faceImg = MakeGO("Face", tv.transform).Add
Component<Image>();
            faceImg.preserveAspect = true;
        }
        faceImg.sprite = GetFaceSprite(t.tileType);
        faceImg.raycastTarget = false;
        tv.bodyImage = bodyImg;
        tv.faceImage = faceImg;
        tv.backImage = FindDeep(tv.transform, "Back")?
.GetComponent<Image>();
        tv.shadowImage = FindDeep(tv.transform, "Shado
w")?.GetComponent<Image>();
        tv.point = FindDeep(tv.transform, "Point");
        tv.SetBaseColor(bodyImg != null ? bodyImg.colo
r : Color.white); // 记住预制体底色，选中取消后恢复
        tv.onClick = OnTileClicked;
        // 图鉴等级 -> 等级背面图 levelN（不同等级花色 Back 颜色不同，共 4 种
)
        int lvl = LevelOf(t.tileType < _codexCollected
.Length ? _codexCollected[t.tileType] : 0);
        Sprite lvlBack = (lvl >= 0 && lvl < _levelBack
s.Length) ? _levelBacks[lvl] : null;
        // 记录新增子层引用（翻转时由 TileView.ToggleFace 按状态统一设置 s
prite）
        tv.bodySub1 = FindDeep(tv.transform, "Body_1")
?.GetComponent<Image>();
        tv.bodySub2 = FindDeep(tv.transform, "Body_2")
?.GetComponent<Image>();
        tv.backSub1 = FindDeep(tv.transform, "Back_1")
?.GetComponent<Image>();
        tv.backSub2 = FindDeep(tv.transform, "Back_2")
?.GetComponent<Image>();
        // 正面牌底图（front）与等级背面图（levelN = backSprite），供 T
oggleFace 按状态设置各层
        tv.frontSprite = bodyImg != null ? bodyImg.spr
ite : null;
        tv.backSprite = lvlBack;
        tv.ToggleFace(true); // 默认正面：body/body_1/body_
```

```
2=front，back/back_1/back_2=levelN
        // 随机 10% 的牌初始为反面状态（ToggleFace(false) 后 body/b
ody_1/body_2=levelN）
        if (Random.value < 0.1f) tv.SetFaceDownInitial
();
        _tiles[t.id] = tv;
        _placedView.Add((t, tv.gameObject));
    }
    LayoutTiles();
}
// 按当前层偏移(_layerDx/_layerDy)重排版面：只更新坐标与绘制顺序，不重建牌。
// 供滑条实时调用，也供 BuildBoard 首次布局调用；高层牌往右上抬形成 2.5D。
private void LayoutTiles()
{
    if (_placedView.Count == 0) return;
    float tw = _cfg.tile_width;
    float th = _cfg.tile_height;
    float ux = tw / 2f;
    float uy = th / 2f;
    float dx = _layerDx;
    float dy = _layerDy;
    float minX = float.MaxValue, minY = float.MaxValue
;
    var placed = new List<(TileOut t, float px, float
py)>();
    foreach (var (t, _) in _placedView)
    {
        float px = t.x * ux + t.layer * dx;
        float py = t.y * uy - t.layer * dy;
        if (px < minX) minX = px;
        if (py < minY) minY = py;
        placed.Add((t, px, py));
    }
    float maxRight = 0, maxBottom = 0;
    foreach (var p in placed)
    {
        float bx = p.px - minX;
        float by = p.py - minY;
        maxRight = Mathf.Max(maxRight, bx + tw);
        maxBottom = Mathf.Max(maxBottom, by + th);
    }
    _nativeW = maxRight;
    _nativeH = maxBottom;
    // 绘制顺序（决定 UGUI 遮挡 / sibling index）：
    // 1) 先按层高 layer 升序 —— 底层先画，高层整体盖在低层之上（2.5D 叠层 z 轴
）；
    // 2) 同层内按网格阅读顺序：先从上到下（py 小=屏幕上方先画），再从左到右（px 小先画）。
    //  下方、右侧的牌后画=覆盖在上，因此右下方向的牌压住左上方向牌的阴影（阴影在右下）。
    placed.Sort((a, b) =>
    {
        int la = a.t.layer, lb = b.t.layer;
        if (la != lb) return la.CompareTo(lb);
        float ay = a.py - minY;  // 布局 y（下为正），小=屏幕上方
        float by = b.py - minY;
        if (ay != by) return ay.CompareTo(by); // 上方先画
，下方后画 → 下压上
        float ax = a.px - minX;  // 布局 x（右为正），小=左
        float bx = b.px - minX;
        return ax.CompareTo(bx);  // 左侧先画，右侧后画 → 右压左
    });
    int span = placed.Count + 1;
    var layerCounter = new Dictionary<int, int>();
    foreach (var p in placed)
    {
        float bx = p.px - minX;
        float by = p.py - minY;
        float cx = bx + tw / 2f - _nativeW / 2f;
```

```
        float cy = -(by + th / 2f) + _nativeH / 2f;
        var go = _tiles[p.t.id].gameObject;
        var tr = go.GetComponent<RectTransform>();
        tr.anchoredPosition = new Vector2(cx, cy);
        var tvh = go.GetComponent<TileView>();
        if (tvh != null) tvh.homePos = new Vector2(cx,
 cy); // 记录原位供抬起/放回动画
        // 层号越大 → sibling index 越大 → UGUI 越后绘制（覆盖在低层之上
) 。
        int ord = layerCounter.TryGetValue(p.t.layer,
 out int o) ? o : 0;
        layerCounter[p.t.layer] = ord + 1;
        tr.SetSiblingIndex(p.t.layer * span + ord);
    }
    FitBoard();
}
private void FitBoard()
{
    _lastScreenW = Screen.width;
    _lastScreenH = Screen.height;
    if (_boardRoot == null) return;
    // 固定缩放 3（不再按屏幕自适应），牌阵整体放大 3 倍
    _boardRoot.localScale = new Vector3(3f, 3f, 1f);
    _boardRoot.anchoredPosition = new Vector2(0, (96f
 - 80f) / 2f);
}
// ---------- 事件处理 ----------
// 点击编排：第一张只变色（不移动）；第二张也只变色；相同则执行碰撞消除；
// 不同或第二张不可点（不符合规则）则前一张取消变色。
private void OnTileClicked(string id)
{
    if (_busy || _state != GameState.Playing || _board
 == null) return;
    if (!_board.IsFree(id)) { OnPickBlocked(id); retur
n; }
    var tv = _tiles[id];
    // 反面牌：先翻转为正面（0.5s），翻转期间锁输入；完成后再走正常选牌流程
    if (!tv.IsFaceUp())
    {
        _busy = true;
        tv.FlipToFront(_flipDur, () => { _busy = false
; if (_tiles.ContainsKey(id)) OnTileClicked(id); });
        return;
    }
    if (_firstPick == "")
    {
        _firstPick = id;
        SelectTile(id);
        PlaySelect0(); // 点击有效牌：播放选中音效
        return;
    }
    if (_firstPick == id) { DeselectTile(id); RevertIf
FaceDown(id); _firstPick = ""; return; }
    string a = _firstPick, b = id;
    if (_board.GetFace(a) == _board.GetFace(b))
    {
        _firstPick = "";
        DeselectTile(a); // 选中牌配对成功，先恢复白色再播放碰撞动画
        DeselectTile(b);
        PlaySelect1(); // 配对成功的第二张牌：播放选中音效
        StartCrash(a, b);
    }
    else
    {
        DeselectTile(a);
        RevertIfFaceDown(a);
        _firstPick = b;
```

```
        SelectTile(b);
        PlaySelect0(); // 第二张也是有效牌（成为新第一张），播放选中音效
    }
}
// 若本是反面牌（被翻转成正面选中），取消选中时切回反面状态
private void RevertIfFaceDown(string id)
{
    if (_tiles.ContainsKey(id))
    {
        var tv = _tiles[id];
        if (tv.wasFaceDown && tv.IsFaceUp()) tv.FlipTo
Back(_flipDur);
    }
}
private void SelectTile(string id) { if (_tiles.Contai
nsKey(id)) _tiles[id].SetSelected(true); }
private void DeselectTile(string id) { if (_tiles.Cont
ainsKey(id)) _tiles[id].SetSelected(false); }
private void ReturnFirst() { if (_firstPick != "") { R
evertIfFaceDown(_firstPick); DeselectTile(_firstPick); _fi
rstPick = ""; } }
// 第二张不符合点击规则：前面的牌取消变色，当前牌闪红提示
private void OnPickBlocked(string id)
{
    ReturnFirst();
    PlayFail(); // 点击无效牌：播放失败音效
    if (_tiles.ContainsKey(id)) StartCoroutine(FlashBl
ocked(_tiles[id]));
}
// 配对成功：抬起 -> 移到中央点左右目标点 -> 一起相撞 -> 音效+缩小 -> 飞向图鉴按钮回收
private void StartCrash(string a, string b)
{
    _busy = true;
    _board.RemovePair(a, b); // 清数据 + 触发 OnMatched（计分
+ 图鉴收集）
    StartCoroutine(CrashSequence(a, b));
}
private IEnumerator CrashSequence(string a, string b)
{
    var ta = _tiles[a];
    var tb = _tiles[b];
    ta.transform.SetAsLastSibling(); // 提到最高层，避免被其它牌遮挡
（UGUI 同级后绘制者在上）
    tb.transform.SetAsLastSibling();
    var ra = ta.GetComponent<RectTransform>().anchored
Position;
    var rb = tb.GetComponent<RectTransform>().anchored
Position;
    var mid = (ra + rb) * 0.5f;
    float X = (_cfg != null) ? _cfg.crashSideOffset :
40f;
    var L = mid + new Vector2(-X, 0);
    var R = mid + new Vector2(X, 0);
    bool aLeft = ra.x <= rb.x;
    // 目标点约束：左目标点不得比左侧牌更靠左、右目标点不得比右侧牌更靠右，
    // 避免牌反向越过自身（左侧牌在左目标点右侧时，左目标点取左侧牌的 X；右侧同理）
    float leftX = aLeft ? ra.x : rb.x;
    float rightX = aLeft ? rb.x : ra.x;
    if (leftX < L.x) L.x = leftX-15f;
    if (rightX > R.x) R.x = rightX+15f;
    var aTarget = aLeft ? L : R;
    var bTarget = aLeft ? R : L;
    float liftDur = _cfg != null ? _cfg.liftDur : 0.15
f;
    float collideDur = _cfg != null ? _cfg.collideDur
: 0.14f;
    float shrinkScale = _cfg != null ? _cfg.shrinkScal
```

```
e : 0.3f;
    float shrinkDur = _cfg != null ? _cfg.shrinkDur :
0.14f;
    float flyDur = _cfg != null ? _cfg.flyDur : 0.45f;
    float pauseDur = _cfg != null ? _cfg.pauseDur : 0.
5f;
    // 阶段1：抬起（隐藏阴影）并移到各自左右目标点
    ta.Lift(); ta.MoveTo(aTarget, liftDur);
    tb.Lift(); tb.MoveTo(bTarget, liftDur);
    yield return new WaitForSeconds(liftDur);
    yield return new WaitForSeconds(pauseDur); // 移到目标
点后停顿，再相撞
    // 阶段2：一起相撞到中央点（严格等待两牌移动协程完成，时长 = collideDur）
    bool ca2 = false, cb2 = false;
    ta.MoveTo(mid, collideDur, () => ca2 = true);
    tb.MoveTo(mid, collideDur, () => cb2 = true);
    yield return new WaitUntil(() => ca2 && cb2);
    // 阶段3：碰撞瞬间播放音效并平滑缩小
    PlayPeng();
    ta.TweenScale(shrinkScale, shrinkDur);
    tb.TweenScale(shrinkScale, shrinkDur);
    yield return new WaitForSeconds(shrinkDur);
    // 阶段4：缩小后飞向图鉴按钮位置后回收（对象池）
    Vector2 codexLocal = _boardRoot.InverseTransformPo
int(_codexBtn.transform.position);
    ta.MoveTo(codexLocal, flyDur, () => ReleaseTileByI
d(a));
    tb.MoveTo(codexLocal, flyDur, () => ReleaseTileByI
d(b));
    yield return new WaitForSeconds(flyDur);
    if (_board.IsWin()) OnWin();
    else if (!_board.HasValidMove()) OnDeadlock();
    if (_state == GameState.Playing) _busy = false;
}
private void OnSelectionChanged(string id, bool on)
{
    if (_tiles.ContainsKey(id)) _tiles[id].SetSelected
(on);
}
private void OnMatched(string a, string b)
{
    float now = Time.time;
    if (_combo > 0 && (now - _lastMatchTime) <= COMBO_
SEC) _combo++;
    else _combo = 1;
    _lastMatchTime = now;
    int mult = Mathf.Min(_combo, 5);
    int gain = BASE_SCORE * mult;
    _score += gain;
    if (_score > _best) { _best = _score; SaveManager.
SaveBest(_best); }
    UpdateHud();
    ShowToast("+" + gain + " 连击x" + mult);
    AddCodex(_board.GetFace(a), 2); // 收集 2 张同花色牌
}
private void OnBlocked(string id)
{
    if (!_tiles.ContainsKey(id)) return;
    var tv = _tiles[id];
    StartCoroutine(FlashBlocked(tv));
}
private void OnWin()
{
    _busy = true;
    _score += ROUND_BONUS;
    if (_score > _best) { _best = _score; SaveManager.
SaveBest(_best); }
```

```
    SaveManager.SaveLevelBest(_currentLevel, _score);
    if (_currentLevel + 1 < _levels.Length) SaveManage
r.UnlockLevel(_currentLevel + 1);
    UpdateHud();
    ShowResult(true);
}
// 卡死（无解）时不直接判失败：弹提示并允许玩家洗牌（救活）或重开本关
private void OnDeadlock()
{
    if (_deadlockOverlay != null) return;  // 已弹提示，避免
重复
    ShowDeadlockPrompt();
}
private void ShowDeadlockPrompt()
{
    if (_uiRoot == null) { ShowToast("已无可消除，请洗牌或重开");
return; }
    var ov = UiTools.MakePanel("DeadlockOverlay");
    ov.transform.SetParent(_uiRoot, false);
    // 全屏半透明 Image 默认拦截射线，提示框显示期间底层不可点，只能操作框内按钮
    UiTools.AddText(ov.transform, "已无可消除的牌，卡死了！\n请点【洗牌
】重排或【重开】本关", 28, Color.white);
    var row = UiTools.MakeGO("Btns", ov.transform);
    var hlg = row.AddComponent<HorizontalLayoutGroup>(
);
    hlg.spacing = 24; hlg.childAlignment = TextAnchor.
MiddleCenter;
    var bShuffle = UiTools.AddButton(row.transform, "洗
牌");
    var leS = bShuffle.gameObject.AddComponent<LayoutE
lement>();
    leS.minHeight = 64; leS.preferredHeight = 64; leS.
flexibleWidth = 1;
    bShuffle.onClick.AddListener(() => { CloseDeadlock
(); _busy = false; DoShuffle(); });
    var bRestart = UiTools.AddButton(row.transform, "重
开");
    var leR = bRestart.gameObject.AddComponent<LayoutE
lement>();
    leR.minHeight = 64; leR.preferredHeight = 64; leR.
flexibleWidth = 1;
    bRestart.onClick.AddListener(() => { CloseDeadlock
(); _busy = false; OnRestart(); });
    _deadlockOverlay = ov;
}
private void CloseDeadlock()
{
    if (_deadlockOverlay != null) { Destroy(_deadlockO
verlay); _deadlockOverlay = null; }
}
// ---------- 提示 / 洗牌 / 重开 ----------
private void OnHint()
{
    if (_busy || _board == null) return;
    if (_hints <= 0) { ShowToast("提示次数用尽"); return; }
    var pair = FindHintPair();
    if (pair.Count == 0) { ShowToast("当前无可消对子"); retur
n; }
    _hints--;
    UpdateHud();
    foreach (var id in pair) if (_tiles.ContainsKey(id
)) _tiles[id].SetHint(true);
    StartCoroutine(ClearHint(pair));
}
private List<string> FindHintPair()
{
    var free = new List<string>();
```

```
    foreach (var id in _board.PresentIds()) if (_board
.IsFree(id)) free.Add(id);
    var byFace = new Dictionary<int, string>();
    foreach (var id in free)
    {
        int f = _board.GetFace(id);
        if (byFace.ContainsKey(f)) return new List<str
ing> { byFace[f], id };
        byFace[f] = id;
    }
    return new List<string>();
}
private void OnShuffle()
{
    if (_busy || _board == null) return;
    DoShuffle();
}
private void DoShuffle()
{
    _busy = true;
    ReturnFirst();
    _board.ClearSelection();
    foreach (var kv in _tiles) kv.Value.SetSelected(fa
lse);
    var ids = _board.PresentIds();
    var faces = new List<int>();
    foreach (var id in ids) faces.Add(_board.GetFace(i
d));
    Shuffle(faces);
    for (int i = 0; i < ids.Count; i++) _board.SetFace
(ids[i], faces[i]);
    if (!_board.HasValidMove())
    {
        var free = new List<string>();
        foreach (var id in ids) if (_board.IsFree(id))
 free.Add(id);
        if (free.Count >= 2) _board.SetFace(free[1], _
board.GetFace(free[0]));
    }
    foreach (var id in ids)
    {
        if (!_tiles.ContainsKey(id)) continue;
        int fi = _board.GetFace(id);
        _tiles[id].SetFaceSprite(GetFaceSprite(fi));
    }
    ShowToast("已洗牌");
    _busy = false;
}
private void OnRestart()
{
    if (_busy) return;
    StartLevel(_currentLevel);
}
private static void Shuffle(List<int> arr)
{
    var rnd = new System.Random();
    for (int i = arr.Count - 1; i > 0; i--)
    {
        int j = rnd.Next(i + 1);
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }
}
// ---------- 协程动画 ----------
private IEnumerator RemoveTile(string id)
{
```

```
    if (!_tiles.ContainsKey(id)) yield break;
    var tv = _tiles[id];
    _tiles.Remove(id);
    var rt = tv.GetComponent<RectTransform>();
    float dur = 0.18f;
    float t = 0;
    while (t < dur)
    {
        t += Time.deltaTime;
        float k = 1f - t / dur;
        rt.localScale = new Vector3(k, k, 1);
        tv.SetAlpha(k);
        yield return null;
    }
    Destroy(tv.gameObject);
}
private IEnumerator FlashBlocked(TileView tv)
{
    Color red = new Color(1f, 0.4f, 0.4f);
    if (tv.bodyImage != null) tv.bodyImage.color = red
;
    if (tv.bodySub1 != null) tv.bodySub1.color = red;
    if (tv.bodySub2 != null) tv.bodySub2.color = red;
    yield return new WaitForSeconds(0.25f);
    if (tv != null && tv.bodyImage != null) tv.bodyIma
ge.color = Color.white;
    if (tv != null && tv.bodySub1 != null) tv.bodySub1
.color = Color.white;
    if (tv != null && tv.bodySub2 != null) tv.bodySub2
.color = Color.white;
}
private IEnumerator ClearHint(List<string> pair)
{
    yield return new WaitForSeconds(1.2f);
    foreach (var id in pair) if (_tiles.ContainsKey(id
)) _tiles[id].SetHint(false);
}
private IEnumerator WaitThenNextRound()
{
    yield return new WaitForSeconds(1.2f);
    StartRound();
    _busy = false;
}
// ---------- HUD / Toast ----------
private void UpdateHud()
{
    if (_scoreLabel != null) _scoreLabel.text = "分数 "
+ _score;
    if (_bestLabel != null) _bestLabel.text = "最高 " +
_best;
    if (_roundLabel != null) _roundLabel.text = (_leve
ls != null && _currentLevel >= 0 && _currentLevel < _level
s.Length) ? _levels[_currentLevel].name : "第 " + _round +
" 局";
    if (_hintBtn != null)
    {
        var txt = _hintBtn.GetComponentInChildren<Text
>();
        if (txt != null) txt.text = "提示(" + _hints + "
)";
    }
}
private void ShowToast(string msg)
{
    if (_toastGO == null || _toast == null) return;
    _toast.text = msg;
    _toastGO.SetActive(true);
```

```
    StopCoroutine("HideToast");
    StartCoroutine("HideToast");
}
private IEnumerator HideToast()
{
    yield return new WaitForSeconds(1.2f);
    if (_toastGO != null) _toastGO.SetActive(false);
}
// ---------- 状态机 / 面板 ----------
private void SetGameUiVisible(bool v)
{
    if (_topBar != null) _topBar.SetActive(v);
    if (_bottomBar != null) _bottomBar.SetActive(v);
    if (_boardRoot != null) _boardRoot.gameObject.SetA
ctive(v);
}
private void HideAllPanels()
{
    SetPanelVisible(_menuPanel, false);
    SetPanelVisible(_levelPanel, false);
    SetPanelVisible(_pausePanel, false);
    SetPanelVisible(_resultPanel, false);
    SetPanelVisible(_settingsPanel, false);
    SetPanelVisible(_codexPanel, false);  // 修复：图鉴此前漏
隐藏，导致开局就显示
}
// 用 CanvasGroup 的 alpha 控制面板显隐（而非 SetActive），便于淡入淡出且对
象保持存活
private void SetPanelVisible(GameObject panel, bool vi
sible)
{
    if (panel == null) return;
    var cg = panel.GetComponent<CanvasGroup>();
    if (cg == null) { panel.SetActive(visible); return
; }
    cg.alpha = visible ? 1 : 0;
    cg.interactable = visible;
    cg.blocksRaycasts = visible;
}
// 确保面板根有 CanvasGroup；缺失时补加（不覆盖用户已调整的 prefab），并初始隐藏
private static void EnsureCanvasGroup(GameObject panel
)
{
    if (panel == null) return;
    var cg = panel.GetComponent<CanvasGroup>();
    if (cg == null) cg = panel.AddComponent<CanvasGrou
p>();
    cg.alpha = 0; cg.interactable = false; cg.blocksRa
ycasts = false;
}
// 黑色蒙版：所有界面切换时黑入/黑出过渡。挂在 Canvas 下并置顶，盖住全部 UI（含 HUD/棋盘
）
private void EnsureFade()
{
    if (_fade != null) return;
    var canvas = GetComponentInParent<Canvas>();
    var parent = (canvas != null) ? canvas.transform :
 (_uiRoot != null ? _uiRoot : transform);
    _fade = new GameObject("FadeMask", typeof(RectTran
sform));
    _fade.transform.SetParent(parent, false);
    var tr = _fade.GetComponent<RectTransform>();
    tr.anchorMin = Vector2.zero; tr.anchorMax = Vector
2.one; tr.offsetMin = Vector2.zero; tr.offsetMax = Vector2
.zero;
    var img = _fade.AddComponent<Image>();
    img.color = Color.black; img.raycastTarget = true;
```

```
    _fadeCg = _fade.AddComponent<CanvasGroup>();
    _fadeCg.alpha = 0; _fadeCg.interactable = false; _
fadeCg.blocksRaycasts = false;
    _fade.transform.SetAsLastSibling();  // 置顶，覆盖所有界面
}
// 界面切换：黑入 → 执行 swap（切换面板/状态）→ 黑出。过渡期间蒙版拦截点击，避免打断
private void DoTransition(System.Action swap)
{
    if (_transitioning) { swap?.Invoke(); return; }
    StartCoroutine(Transition(swap));
}
private System.Collections.IEnumerator Transition(Syst
em.Action swap)
{
    _transitioning = true;
    if (_fadeCg != null) _fadeCg.blocksRaycasts = true
;
    yield return FadeTo(1f, 0.18f);
    swap?.Invoke();
    yield return new WaitForSeconds(0.02f);
    yield return FadeTo(0f, 0.22f);
    if (_fadeCg != null) _fadeCg.blocksRaycasts = fals
e;
    _transitioning = false;
}
private System.Collections.IEnumerator FadeTo(float ta
rget, float dur)
{
    if (_fadeCg == null) yield break;
    float a = _fadeCg.alpha;
    float t = 0;
    while (t < dur)
    {
        t += Time.deltaTime;
        _fadeCg.alpha = Mathf.Lerp(a, target, Mathf.Cl
amp01(t / dur));
        yield return null;
    }
    _fadeCg.alpha = target;
}
private GameObject MakePanel(Transform parent, string
name)
{
    var go = MakeGO(name, parent);
    var tr = go.GetComponent<RectTransform>();
    tr.anchorMin = Vector2.zero; tr.anchorMax = Vector
2.one;
    tr.offsetMin = Vector2.zero; tr.offsetMax = Vector
2.zero;
    var img = go.AddComponent<Image>();
    img.color = new Color(0f, 0f, 0f, 0.7f);
    var lg = go.AddComponent<VerticalLayoutGroup>();
    lg.spacing = 20; lg.childAlignment = TextAnchor.Mi
ddleCenter;
    lg.padding = new RectOffset(40, 40, 40, 40);
    return go;
}
private void PanelButton(GameObject panel, string labe
l, UnityEngine.Events.UnityAction cb)
{
    var b = AddButton(panel, label);
    var le = b.gameObject.AddComponent<LayoutElement>(
);
    le.minHeight = 72; le.preferredHeight = 72; le.fle
xibleWidth = 1;
    b.onClick.AddListener(cb);
}
```

```
// ---------- 设置界面（参考设计稿：音乐/音效/震动开关 + 画质三档 + 语音 + 返回）
----------
private void ShowSettings()
{
    if (_settingsPanel == null) return;
    HideAllPanels();
    SetPanelVisible(_settingsPanel, true);
}
// 加载设置面板预制体并实例化显示（预制体需由 Tools/生成设置面板预制体 生成；UI 已内置于 pr
efab）
private void LoadSettingsPrefab(Transform parent)
{
    var prefab = Resources.Load<GameObject>("Prefab/Se
ttingsPanel");
    if (prefab == null)
    {
        Debug.LogError("[Settings] 未找到 Resources/Setti
ngsPanel.prefab，请运行菜单 Tools / 生成设置面板预制体");
        _settingsPanel = null;
        return;
    }
    _settingsPanel = Instantiate(prefab, parent);
    EnsureCanvasGroup(_settingsPanel);
    var sp = _settingsPanel.GetComponent<SettingsPanel
>();
    if (sp != null) sp.Init(_settings,
        onBack: ShowMenu,
        onAudio: ApplyAudioSettings,
        onQuality: ApplyQualitySetting,
        onSave: () => SaveManager.SaveSettings(_settin
gs));
}
private void BuildPanels(Transform parent)
{
    _uiRoot = parent;
    LoadMenuPrefab(parent);
    LoadLevelPrefab(parent);
    LoadPausePrefab(parent);
    LoadResultPrefab(parent);
    LoadCodexPrefab(parent);
    LoadSettingsPrefab(parent);
    HideAllPanels();
    EnsureFade();
    SetPanelVisible(_menuPanel, true);
}
private void LoadMenuPrefab(Transform parent)
{
    var prefab = Resources.Load<GameObject>("Prefab/Me
nuPanel");
    if (prefab == null) { Debug.LogError("[UI] 未找到 Pre
fab/MenuPanel，请运行 Tools/生成 UI 预制体"); _menuPanel = null; re
turn; }
    _menuPanel = Instantiate(prefab, parent);
    EnsureCanvasGroup(_menuPanel);
    _menuPanelC = _menuPanel.GetComponent<MenuPanel>()
;
    if (_menuPanelC != null) _menuPanelC.Init(
        () => DoTransition(() => StartLevel(Mathf.Min(
SaveManager.LoadMaxUnlocked(), Mathf.Max(0, _levels.Length
 - 1)))),
        ShowLevelSelect, ShowSettings, ShowCodex);
}
private void LoadLevelPrefab(Transform parent)
{
    var prefab = Resources.Load<GameObject>("Prefab/Le
velPanel");
    if (prefab == null) { Debug.LogError("[UI] 未找到 Pre
```

```
fab/LevelPanel，请运行 Tools/生成 UI 预制体"); _levelPanel = null;
return; }
    _levelPanel = Instantiate(prefab, parent);
    EnsureCanvasGroup(_levelPanel);
    _levelPanelC = _levelPanel.GetComponent<LevelSelec
tPanel>();
    if (_levelPanelC != null) _levelPanelC.Init(ShowMe
nu);
}
private void LoadPausePrefab(Transform parent)
{
    var prefab = Resources.Load<GameObject>("Prefab/Pa
usePanel");
    if (prefab == null) { Debug.LogError("[UI] 未找到 Pre
fab/PausePanel，请运行 Tools/生成 UI 预制体"); _pausePanel = null;
return; }
    _pausePanel = Instantiate(prefab, parent);
    EnsureCanvasGroup(_pausePanel);
    _pausePanelC = _pausePanel.GetComponent<PausePanel
>();
    if (_pausePanelC != null) _pausePanelC.Init(OnResu
me, OnRetry, OnToMenu);
}
private void LoadResultPrefab(Transform parent)
{
    var prefab = Resources.Load<GameObject>("Prefab/Re
sultPanel");
    if (prefab == null) { Debug.LogError("[UI] 未找到 Pre
fab/ResultPanel，请运行 Tools/生成 UI 预制体"); _resultPanel = null
; return; }
    _resultPanel = Instantiate(prefab, parent);
    EnsureCanvasGroup(_resultPanel);
    _resultPanelC = _resultPanel.GetComponent<ResultPa
nel>();
    if (_resultPanelC != null) _resultPanelC.Init(OnNe
xtLevel, OnRetry, OnToMenu, () => StartLevel(0));
}
private void LoadCodexPrefab(Transform parent)
{
    var prefab = Resources.Load<GameObject>("Prefab/Co
dexPanel");
    if (prefab == null) { Debug.LogError("[UI] 未找到 Pre
fab/CodexPanel，请运行 Tools/生成 UI 预制体"); _codexPanel = null;
return; }
    _codexPanel = Instantiate(prefab, parent);
    EnsureCanvasGroup(_codexPanel);
    _codexPanelC = _codexPanel.GetComponent<CodexPanel
>();
    if (_codexPanelC != null) _codexPanelC.Init(CloseC
odex);
}
private void ShowMenu()
{
    _state = GameState.Menu;
    HideAllPanels();
    SetGameUiVisible(false);
    SetPanelVisible(_menuPanel, true);
}
private void ShowLevelSelect()
{
    _state = GameState.LevelSelect;
    HideAllPanels();
    SetGameUiVisible(false);
    if (_levelPanelC != null)
    {
        string[] names = new string[_levels.Length];
        for (int i = 0; i < _levels.Length; i++) names
```

```
[i] = _levels[i].name;
        _levelPanelC.BuildList(names, SaveManager.Load
MaxUnlocked(), idx => DoTransition(() => StartLevel(idx)))
;
    }
    SetPanelVisible(_levelPanel, true);
}
private void StartLevel(int idx)
{
    if (_levels == null || idx < 0 || idx >= _levels.L
ength) { ShowMenu(); return; }
    _currentLevel = idx;
    _layoutData = _levels[idx].layout;
    _score = 0;
    _state = GameState.Playing;
    HideAllPanels();
    SetGameUiVisible(true);
    StartRound();
}
private void OnPause()
{
    if (_state != GameState.Playing) return;
    _state = GameState.Paused;
    SetPanelVisible(_pausePanel, true);
}
private void OnResume()
{
    _state = GameState.Playing;
    SetPanelVisible(_pausePanel, false);
}
private void OnToMenu()
{
    ShowMenu();
}
private void OnRetry()
{
    StartLevel(_currentLevel);
}
private void OnNextLevel()
{
    if (_currentLevel + 1 < _levels.Length) DoTransiti
on(() => StartLevel(_currentLevel + 1));
    else ShowMenu();
}
private void ShowResult(bool win)
{
    _state = GameState.Result;
    bool hasNext = win && (_currentLevel + 1 < _levels
.Length);
    bool isLast = win && (_currentLevel + 1 >= _levels
.Length);
    string info = "得分 " + _score + "\n最高 " + _best;
    if (_levels != null && _currentLevel >= 0 && _curr
entLevel < _levels.Length)
        info += "\n" + _levels[_currentLevel].name;
    if (!win) info += "\n无解，本关失败";
    if (_resultPanelC != null) _resultPanelC.Show(win,
 info, hasNext, isLast);
    HideAllPanels();
    SetGameUiVisible(false);
    SetPanelVisible(_resultPanel, true);
}
// ---------- 关卡加载（Resources/level/levelN.json） ------
----
private class LevelEntry
{
    public string name;
```

```
    public int targetScore;
    public int seed;
    public LayoutData layout;
}
private void LoadLevels()
{
    var assets = Resources.LoadAll<TextAsset>("level")
;
    if (assets == null || assets.Length == 0)
    {
        Debug.LogError("GameController: 未找到 Resources/
level/ 下的关卡文件");
        _levels = new LevelEntry[0];
        return;
    }
    var parsed = new List<(int order, LevelFile f)>();
    foreach (var a in assets)
    {
        var f = JsonUtility.FromJson<LevelFile>(a.text
);
        if (f == null) continue;
        parsed.Add((ExtractLevelOrder(a.name), f));
    }
    parsed.Sort((x, y) => x.order.CompareTo(y.order));
    var list = new List<LevelEntry>();
    foreach (var (order, f) in parsed)
    {
        var e = new LevelEntry();
        e.name = string.IsNullOrEmpty(f.name) ? ("第" +
 order + "关") : f.name;
        e.targetScore = f.targetScore;
        e.seed = f.seed;
        if (f.tiles != null && f.tiles.Length > 0)
            e.layout = new LayoutData { tiles = f.tile
s };
        else
            e.layout = LayoutFactory.MakeTower(f.cols,
 f.rows, f.layers);
        list.Add(e);
    }
    _levels = list.ToArray();
}
private static int ExtractLevelOrder(string fileName)
{
    int n = 0;
    foreach (char c in fileName)
        if (c >= '0' && c <= '9') n = n * 10 + (c - '0
');
    return n;
}
// 递归查找子物体（Transform.Find 只查直接子级，预制体新增嵌套容器后会找不到）
private static Transform FindDeep(Transform root, stri
ng name)
{
    foreach (Transform c in root)
    {
        if (c.name == name) return c;
        var d = FindDeep(c, name);
        if (d != null) return d;
    }
    return null;
}
private static void SetChildSprite(Transform root, str
ing name, Sprite s)
{
    if (s == null) return;
    var img = FindDeep(root, name)?.GetComponent<Image
```

```
>();
    if (img != null) img.sprite = s;
}
private static void SetChildColor(Transform root, stri
ng name, Color c)
{
    var img = FindDeep(root, name)?.GetComponent<Image
>();
    if (img != null) img.color = c;
}
// 自建最小可用 Slider（不依赖任何预制体），用于调试层偏移等实时调参
private Slider AddSlider(Transform parent, string name
, float min, float max, float val, UnityEngine.Events.Unit
yAction<float> cb)
{
    var go = MakeGO(name, parent);
    var tr = go.GetComponent<RectTransform>();
    tr.sizeDelta = new Vector2(240, 36);
    var bg = MakeGO("Background", go.transform);
    var bgr = bg.GetComponent<RectTransform>();
    bgr.anchorMin = Vector2.zero; bgr.anchorMax = Vect
or2.one; bgr.offsetMin = Vector2.zero; bgr.offsetMax = Vec
tor2.zero;
    var bgImg = bg.AddComponent<Image>(); bgImg.color
= new Color(1f, 1f, 1f, 0.18f);
    var fillArea = MakeGO("FillArea", go.transform);
    var far = fillArea.GetComponent<RectTransform>();
    far.anchorMin = new Vector2(0f, 0.25f); far.anchor
Max = new Vector2(1f, 0.75f); far.offsetMin = Vector2.zero
; far.offsetMax = Vector2.zero;
    var fill = MakeGO("Fill", fillArea.transform);
    var filr = fill.GetComponent<RectTransform>();
    filr.anchorMin = Vector2.zero; filr.anchorMax = Ve
ctor2.one; filr.offsetMin = Vector2.zero; filr.offsetMax =
 Vector2.zero;
    var fillImg = fill.AddComponent<Image>(); fillImg.
color = new Color(1f, 0.84f, 0.40f);
    var handle = MakeGO("Handle", go.transform);
    var hr = handle.GetComponent<RectTransform>();
    hr.sizeDelta = new Vector2(24, 36); hr.anchorMin =
 new Vector2(0f, 0.5f); hr.anchorMax = new Vector2(0f, 0.5
f); hr.pivot = new Vector2(0.5f, 0.5f);
    var hImg = handle.AddComponent<Image>(); hImg.colo
r = Color.white;
    var slider = go.AddComponent<Slider>();
    slider.minValue = min; slider.maxValue = max; slid
er.value = val;
    slider.fillRect = filr;
    slider.handleRect = hr;
    slider.targetGraphic = hImg;
    slider.onValueChanged.AddListener(cb);
    return slider;
}
// ---------- 对象池 ----------
private TileView GetTile()
{
    foreach (var p in _pool) if (p != null && !p.gameO
bject.activeSelf) { p.gameObject.SetActive(true); return p
; }
    GameObject go = _tilePrefab != null ? Instantiate(
_tilePrefab, _boardRoot) : MakeGO("Tile", _boardRoot);
    var tv = go.GetComponent<TileView>();
    if (tv == null) tv = go.AddComponent<TileView>();
    return tv;
}
private void ReleaseTile(TileView tv)
{
```

```
    if (tv == null) return;
    tv.gameObject.SetActive(false);
    if (!_pool.Contains(tv)) _pool.Add(tv);
}
private void ReleaseTileById(string id)
{
    if (!_tiles.ContainsKey(id)) return;
    var tv = _tiles[id];
    _tiles.Remove(id);
    _placedView.RemoveAll(x => x.go == tv.gameObject);
    ReleaseTile(tv);
}
private void PlayPeng()
{
    if (_audio != null && _peng != null) _audio.PlayOn
eShot(_peng);
    if (_settings != null && _settings.vibration && Ap
plication.isMobilePlatform) Handheld.Vibrate(); // 震动开关（仅移
动端生效）
}
private void PlayFail() { if (_audio != null && _fail
!= null) _audio.PlayOneShot(_fail); }
private void PlaySelect0() { if (_audio != null && _se
lect0 != null) _audio.PlayOneShot(_select0); }
private void PlaySelect1() { if (_audio != null && _se
lect1 != null) _audio.PlayOneShot(_select1); }
// 游戏开始循环播放背景音乐（若已在播放则不重复触发）
private void PlayBgm()
{
    if (_bgmSrc == null || _bgm == null) return;
    if (_bgmSrc.clip != _bgm) { _bgmSrc.clip = _bgm; _
bgmSrc.loop = true; }
    if (!_bgmSrc.isPlaying) _bgmSrc.Play();
}
// 应用音频开关：音乐只作用于 bgm 音源，音效作用于所有 PlayOneShot
private void ApplyAudioSettings()
{
    if (_settings == null) return;
    if (_bgmSrc != null) _bgmSrc.mute = !_settings.mus
ic;
    if (_audio != null) _audio.mute = !_settings.sfx;
}
// 应用画面质量档位（0低/1中/2高）
private void ApplyQualitySetting()
{
    if (_settings == null) return;
    int q = Mathf.Clamp(_settings.quality, 0, 2);
    if (QualitySettings.names != null && QualitySettin
gs.names.Length > 0)
        QualitySettings.SetQualityLevel(Mathf.Min(q, Q
ualitySettings.names.Length - 1), true);
}
// ---------- 图鉴 ----------
private void EnsureCodexSize()
{
    if (_codexCollected.Length == _faceSprites.Count)
return;
    var tmp = new int[_faceSprites.Count];
    for (int i = 0; i < tmp.Length && i < _codexCollec
ted.Length; i++) tmp[i] = _codexCollected[i];
    _codexCollected = tmp;
}
private void AddCodex(int face, int n)
{
    if (face < 0 || face >= _codexCollected.Length) re
turn;
    _codexCollected[face] += n;
```

```
    SaveManager.SaveCodex(_codexCollected);
  }
  private static int LevelOf(int count) => count >= 1000
  ? 3 : (count >= 100 ? 2 : (count >= 10 ? 1 : 0));
  private static Color CodexColor(int lvl) => CODE_COLOR
  S[Mathf.Clamp(lvl, 0, CODE_COLORS.Length - 1)];
  private void ShowCodex()
  {
    if (_state != GameState.Playing && _state != GameS
  tate.Menu) return;
    _codexReturnState = _state;
    _state = GameState.Paused;
    // 从主菜单打开时先隐藏主菜单，避免图鉴面板下透出菜单
    if (_codexReturnState == GameState.Menu && _menuPa
  nel != null) SetPanelVisible(_menuPanel, false);
    if (_codexPanelC != null)
    {
      var faces = new System.Collections.Generic.Lis
  t<Sprite>();
      for (int f = 0; f < _faceSprites.Count; f++) f
  aces.Add(GetFaceSprite(f));
      _codexPanelC.Refresh(faces, _codexCollected, _
  levelBacks, CODE_COLORS);
    }
    SetPanelVisible(_codexPanel, true);
  }
  private void CloseCodex()
  {
    SetPanelVisible(_codexPanel, false);
    _state = _codexReturnState;
    // 从主菜单打开时恢复主菜单显示
    if (_codexReturnState == GameState.Menu && _menuPa
  nel != null) SetPanelVisible(_menuPanel, true);
  }
}
// ===== FILE: Assets\Scripts\UI\GamePanel.cs =====
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
/// <summary>
/// 游戏主界面（HUD）预制体组件，挂在 GamePanel 预制体根。
/// 由 UiPrefabGenerator.BuildGame 生成，字段在生成时直接绑定；Init 绑定按钮/
  滑条回调。
/// GameController 优先从 Prefab/GamePanel 加载此预制体，缺失时回退到运行时 M
  akeGO 构建。
/// </summary>
public class GamePanel : MonoBehaviour
{
  public RectTransform topBar, bottomBar, boardRoot, toa
  st;
  public Text scoreLabel, bestLabel, roundLabel, toastTe
  xt, xLabel, yLabel;
  public Button hintBtn, shuffleBtn, restartBtn, pauseBt
  n, codexBtn;
  public Slider xOffsetSlider, yOffsetSlider;
  public void Init(UnityAction onHint, UnityAction onShu
  ffle, UnityAction onRestart,
            UnityAction onPause, UnityAction onCo
  dex,
            float xMin, float xMax, float xVal, U
  nityAction<float> onX,
            float yMin, float yMax, float yVal, U
  nityAction<float> onY,
            bool showSliders)
  {
    hintBtn?.onClick.AddListener(onHint);
    shuffleBtn?.onClick.AddListener(onShuffle);
```

```
        restartBtn?.onClick.AddListener(onRestart);
        pauseBtn?.onClick.AddListener(onPause);
        codexBtn?.onClick.AddListener(onCodex);
        if (xOffsetSlider != null)
        {
            xOffsetSlider.minValue = xMin; xOffsetSlider.m
    axValue = xMax; xOffsetSlider.value = xVal;
            xOffsetSlider.onValueChanged.AddListener(onX);
        }
        if (yOffsetSlider != null)
        {
            yOffsetSlider.minValue = yMin; yOffsetSlider.m
    axValue = yMax; yOffsetSlider.value = yVal;
            yOffsetSlider.onValueChanged.AddListener(onY);
        }
        SetSlidersVisible(showSliders);
    }
    public void SetSlidersVisible(bool v)
    {
        if (xOffsetSlider != null) xOffsetSlider.gameObjec
    t.SetActive(v);
        if (yOffsetSlider != null) yOffsetSlider.gameObjec
    t.SetActive(v);
        if (xLabel != null) xLabel.gameObject.SetActive(v)
    ;
        if (yLabel != null) yLabel.gameObject.SetActive(v)
    ;
    }
}
// ===== FILE: Assets\Scripts\UI\LevelItem.cs =====
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
/// <summary>
/// 关卡列表项预制体组件（挂在 LevelItem 预制体根，根即 Button）。
/// 预制体结构：根 Button（背景 Image）+ 子 Text。Bind 设置文本 / 解锁态 / 点击回
    调。
/// </summary>
public class LevelItem : MonoBehaviour
{
    private Button _btn;
    private Text _txt;
    private void Awake()
    {
        _btn = GetComponent<Button>();
        _txt = GetComponentInChildren<Text>();
    }
    public void Bind(int levelIndex, string name, bool loc
    ked, UnityAction<int> onPick)
    {
        if (_txt != null) _txt.text = name;
        if (_btn != null)
        {
            _btn.interactable = !locked;
            _btn.onClick.RemoveAllListeners();
            if (!locked) _btn.onClick.AddListener(() => on
    Pick(levelIndex));
        }
    }
}
// ===== FILE: Assets\Scripts\UI\LevelSelectPanel.cs =====
using System;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
using System.Collections.Generic;
/// <summary>
```

```
/// 关卡选择面板组件（挂在 LevelPanel 预制体根）。预制体含：标题 + LevelList(VLG 容
  器) + Btn_Back。
/// Init 绑定返回回调；BuildList 按解锁状态运行时实例化 LevelItem 预制体并 Bind。
/// </summary>
public class LevelSelectPanel : MonoBehaviour
{
  private Transform _list;
  public void Init(UnityAction onBack)
  {
    _list = FindDeep(transform, "LevelList");
    var backBtn = FindDeep(transform, "Btn_Back")?.Get
  Component<Button>();
    if (backBtn == null) backBtn = FindButtonByText(tr
  ansform, "返回");
    backBtn?.onClick.AddListener(onBack);
  }
  public void BuildList(string[] names, int unlockedMax,
   UnityAction<int> onPick)
  {
    if (_list == null) return;
    var children = new List<GameObject>();
    for (int i = 0; i < _list.childCount; i++) childre
  n.Add(_list.GetChild(i).gameObject);
    foreach (var c in children) UnityEngine.Object.Des
  troy(c);
    var prefab = Resources.Load<GameObject>("Prefab/Le
  velItem");
    for (int i = 0; i < names.Length; i++)
    {
      int idx = i;
      bool locked = i > unlockedMax;
      GameObject go;
      if (prefab != null)
      {
        go = Instantiate(prefab, _list);
      }
      else
      {
        // 回退：LevelItem 预制体缺失时运行时构建按钮
        var b = UiTools.AddButton(_list, names[i],
   UiTools.LoadFont());
        go = b.gameObject;
        var le = go.AddComponent<LayoutElement>();
        le.minHeight = 64; le.preferredHeight = 64
  ; le.flexibleWidth = 1;
        b.interactable = !locked;
        if (!locked) b.onClick.AddListener(() => o
  nPick(idx));
        continue;
      }
      var item = go.GetComponent<LevelItem>();
      if (item != null) item.Bind(idx, names[i], loc
  ked, onPick);
      else
      {
        var le = go.AddComponent<LayoutElement>();
        le.minHeight = 64; le.preferredHeight = 64
  ; le.flexibleWidth = 1;
        var b = go.GetComponent<Button>();
        if (b != null) { b.interactable = !locked;
   if (!locked) b.onClick.AddListener(() => onPick(idx)); }
      }
    }
  }
  private static Transform FindDeep(Transform root, stri
  ng name)
  {
```

```
        foreach (Transform c in root)
        {
            if (c.name == name) return c;
            var d = FindDeep(c, name);
            if (d != null) return d;
        }
        return null;
    }
    private static Button FindButtonByText(Transform root,
     string text)
    {
        var btn = root.GetComponent<Button>();
        if (btn != null)
        {
            var t = root.GetComponentInChildren<Text>();
            if (t != null && t.text == text) return btn;
        }
        foreach (Transform c in root)
        {
            var b = FindButtonByText(c, text);
            if (b != null) return b;
        }
        return null;
    }
}
// ===== FILE: Assets\Scripts\UI\MenuPanel.cs =====
using System;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
/// <summary>
/// 主菜单面板组件（挂在 MenuPanel 预制体根）。预制体含：标题 Text + 三个按钮
/// Btn_Start / Btn_Levels / Btn_Settings。Init 绑定回调，不生成 UI
    。
/// </summary>
public class MenuPanel : MonoBehaviour
{
    public void Init(UnityAction onStart, UnityAction onLe
    vels, UnityAction onSettings, UnityAction onCodex)
    {
        transform.Find("Btn_Start")?.GetComponent<Button>(
    )?.onClick.AddListener(onStart);
        transform.Find("Btn_Levels")?.GetComponent<Button>
    ()?.onClick.AddListener(onLevels);
        transform.Find("Btn_Settings")?.GetComponent<Butto
    n>()?.onClick.AddListener(onSettings);
        // 主菜单上的图鉴按钮（用户手动添加，可能嵌套层级）：递归查找 + 文本兜底
        var codexBtn = FindDeep(transform, "Btn_Codex")?.G
    etComponent<Button>();
        if (codexBtn == null) codexBtn = FindButtonByText(
    transform, "图鉴");
        codexBtn?.onClick.AddListener(onCodex);
    }
    private static Transform FindDeep(Transform root, stri
    ng name)
    {
        foreach (Transform c in root)
        {
            if (c.name == name) return c;
            var d = FindDeep(c, name);
            if (d != null) return d;
        }
        return null;
    }
    private static Button FindButtonByText(Transform root,
     string text)
    {
```

```
        var btn = root.GetComponent<Button>();
        if (btn != null)
        {
            var t = root.GetComponentInChildren<Text>();
            if (t != null && t.text == text) return btn;
        }
        foreach (Transform c in root)
        {
            var b = FindButtonByText(c, text);
            if (b != null) return b;
        }
        return null;
    }
}
// ===== FILE: Assets\Scripts\UI\PausePanel.cs =====
using System;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
/// <summary>
/// 暂停面板组件（挂在 PausePanel 预制体根）。预制体含按钮
/// Btn_Resume / Btn_Retry / Btn_Menu。Init 绑定回调，不生成 UI。
/// </summary>
public class PausePanel : MonoBehaviour
{
    public void Init(UnityAction onResume, UnityAction onR
    etry, UnityAction onMenu)
    {
        transform.Find("Btn_Resume")?.GetComponent<Button>
    ()?.onClick.AddListener(onResume);
        transform.Find("Btn_Retry")?.GetComponent<Button>(
    )?.onClick.AddListener(onRetry);
        transform.Find("Btn_Menu")?.GetComponent<Button>()
    ?.onClick.AddListener(onMenu);
    }
}
// ===== FILE: Assets\Scripts\UI\ResultPanel.cs =====
using System;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
/// <summary>
/// 结算面板组件（挂在 ResultPanel 预制体根）。预制体含：Title(Text) + Info(Te
    xt) +
/// Btn_Next(Button) + Btn_Retry(Button) + Btn_Menu(Button
    )。Init 绑定回调；
/// Show 设置文本与按钮显隐。最后一关通关时 Btn_Next 复用为"从头再玩"（循环）。
/// </summary>
public class ResultPanel : MonoBehaviour
{
    private Text _title;
    private Text _info;
    private Button _next;
    private UnityAction _onNext;
    private UnityAction _onReplayAll;
    public void Init(UnityAction onNext, UnityAction onRet
    ry, UnityAction onMenu, UnityAction onReplayAll = null)
    {
        _title = transform.Find("Title")?.GetComponent<Tex
    t>();
        _info = transform.Find("Info")?.GetComponent<Text>
    ();
        _next = transform.Find("Btn_Next")?.GetComponent<B
    utton>();
        _onNext = onNext;
        _onReplayAll = onReplayAll;
        transform.Find("Btn_Retry")?.GetComponent<Button>(
```

```
  )?.onClick.AddListener(onRetry);
    transform.Find("Btn_Menu")?.GetComponent<Button>()
  ?.onClick.AddListener(onMenu);
  }
  // win: 是否过关；hasNext: 是否还有下一关；isLast: 是否最后一关（已通关全部 ->
  循环再玩）
  public void Show(bool win, string info, bool hasNext,
  bool isLast)
  {
    if (_title != null) _title.text = win ? (isLast ?
  "全部通关!" : "过关!") : "失败";
    if (_info != null) _info.text = info;
    if (_next != null)
    {
      bool showNext = win && (hasNext || isLast);
      _next.gameObject.SetActive(showNext);
      _next.onClick.RemoveAllListeners();
      if (showNext)
      {
        var t = _next.GetComponentInChildren<Text>
  ();
        if (t != null) t.text = hasNext ? "下一关" :
  "从头再玩";
        var act = hasNext ? _onNext : _onReplayAll
  ;
        if (act != null) _next.onClick.AddListener
  (act);
      }
    }
  }
}
// ===== FILE: Assets\Scripts\UI\SaveManager.cs =====
using System.IO;
using System.Collections.Generic;
using UnityEngine;
/// <summary>
/// 进度持久化：最高分 / 已解锁关卡 / 每关最佳分，写入 Application.persistentDat
  aPath/save.json。
/// </summary>
public static class SaveManager
{
  private const string FILE = "save.json";
  [System.Serializable]
  private class ProgressData
  {
    public int best = 0;
    public int maxUnlocked = 0;
    public List<int> levelBest = new List<int>();
    public int[] codex; // 每种花色收集数（按 faceIndex），图鉴用
    public SettingsData settings = new SettingsData();
  }
  /// <summary>
  /// 设置界面数据：音乐/音效/震动开关、画面质量档位、语言选择。
  /// </summary>
  [System.Serializable]
  public class SettingsData
  {
    public bool music = true;
    public bool sfx = true;
    public bool vibration = true;
    public int quality = 1;  // 0=低 1=中 2=高
    public int language = 0;  // 0=简体中文 1=English
  }
  public static SettingsData LoadSettings()
  {
    var d = Read();
    if (d.settings == null) d.settings = new SettingsD
```

```
ata();
    return d.settings;
}
public static void SaveSettings(SettingsData s)
{
    var d = Read();
    d.settings = s;
    Write(d);
}
private static ProgressData Read()
{
    string p = Path.Combine(Application.persistentData
Path, FILE);
    if (!File.Exists(p)) return new ProgressData();
    try
    {
        var v = JsonUtility.FromJson<ProgressData>(Fil
e.ReadAllText(p));
        return v ?? new ProgressData();
    }
    catch (System.Exception e)
    {
        Debug.LogWarning("SaveManager.Read 失败: " + e.M
essage);
        return new ProgressData();
    }
}
private static void Write(ProgressData d)
{
    string p = Path.Combine(Application.persistentData
Path, FILE);
    try { File.WriteAllText(p, JsonUtility.ToJson(d));
 }
    catch (System.Exception e) { Debug.LogWarning("Sav
eManager.Write 失败: " + e.Message); }
}
// 兼容旧接口
public static int LoadBest() { return Read().best; }
public static void SaveBest(int best)
{
    var d = Read();
    if (best > d.best) d.best = best;
    Write(d);
}
// 关卡进度
public static int LoadMaxUnlocked() { return Read().ma
xUnlocked; }
public static void UnlockLevel(int idx)
{
    var d = Read();
    if (idx > d.maxUnlocked) d.maxUnlocked = idx;
    Write(d);
}
public static int LoadLevelBest(int idx)
{
    var d = Read();
    return (d.levelBest != null && idx < d.levelBest.C
ount) ? d.levelBest[idx] : 0;
}
public static void SaveLevelBest(int idx, int score)
{
    var d = Read();
    while (d.levelBest.Count <= idx) d.levelBest.Add(0
);
    if (score > d.levelBest[idx]) d.levelBest[idx] = s
core;
    Write(d);
```

```
    }
    // 图鉴：每种花色收集数（按 faceIndex）
    public static int[] LoadCodex()
    {
        var d = Read();
        return (d.codex != null) ? d.codex : new int[0];
    }
    public static void SaveCodex(int[] c)
    {
        var d = Read();
        d.codex = c;
        Write(d);
    }
}
// ===== FILE: Assets\Scripts\UI\SettingsPanel.cs =====
using System;
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// 设置面板组件：挂在 SettingsPanel 预制体根上。
/// 预制体已包含完整 UI 层级（遮罩 / 米色卡片 / 标题 / 音乐 · 音效 · 震动开关 / 画质三档 / 语音
    / 返回）。
/// 本组件只负责查找并绑定这些子对象、回写 SaveManager.SettingsData，不在运行时生成 U
    I。
/// 子对象命名约定（由 SettingsPanelGenerator 生成时保持一致）：
///   SettingsCard
///     Row_音乐 / Row_音效 / Row_震动  -> Track(Button+Image)
    -> Handle(Image)
///     Row_Quality -> Btn_0 / Btn_1 / Btn_2
///     Row_Lang   -> LangBtn(Button)
///   BackBtn
/// </summary>
public class SettingsPanel : MonoBehaviour
{
    private SaveManager.SettingsData _data;
    private Action _onBack;
    private Action _onAudio;
    private Action _onQuality;
    private Action _onSave;
    private Image _musicTrack, _sfxTrack, _vibTrack;
    private RectTransform _musicHandle, _sfxHandle, _vibHa
    ndle;
    private Button[] _qualityBtns = new Button[3];
    private Button _langBtn;
    private Text _langText;
    /// <summary>由 GameController 在实例化预制体后调用：查找子对象并绑定回调。</
    summary>
    public void Init(SaveManager.SettingsData data, Action
     onBack, Action onAudio, Action onQuality, Action onSave)
    {
        _data = data; _onBack = onBack; _onAudio = onAudio
    ; _onQuality = onQuality; _onSave = onSave;
        Bind();
    }
    private void Bind()
    {
        var card = transform.Find("SettingsCard");
        if (card == null) { Debug.LogWarning("[Settings] 预
    制体缺少 SettingsCard，请重新运行 Tools/生成设置面板预制体"); return; }
        (_musicTrack, _musicHandle) = BindToggle(card, "Ro
    w_音乐", () => _data.music, v => { _data.music = v; _onSave(
    ); _onAudio(); });
        (_sfxTrack, _sfxHandle) = BindToggle(card, "Row_音效
    ", () => _data.sfx, v => { _data.sfx = v; _onSave(); _onAu
    dio(); });
        (_vibTrack, _vibHandle) = BindToggle(card, "Row_震动
    ", () => _data.vibration, v => { _data.vibration = v; _onS
```

```
ave(); });
    var qrow = card.Find("Row_Quality");
    if (qrow != null)
    {
        for (int i = 0; i < 3; i++)
        {
            int idx = i;
            _qualityBtns[i] = qrow.Find("Btn_" + i).Ge
tComponent<Button>();
            if (_qualityBtns[i] != null)
                _qualityBtns[i].onClick.AddListener(()
 => { _data.quality = idx; _onSave(); _onQuality(); Refres
hQuality(); });
        }
        RefreshQuality();
    }
    var lrow = card.Find("Row_Lang");
    if (lrow != null)
    {
        _langBtn = lrow.Find("LangBtn").GetComponent<B
utton>();
        _langText = _langBtn != null ? _langBtn.GetCom
ponentInChildren<Text>() : null;
        if (_langBtn != null) _langBtn.onClick.AddList
ener(() => { _data.language = (_data.language + 1) % 2; _o
nSave(); RefreshLang(); });
        RefreshLang();
    }
    var back = transform.Find("BackBtn");
    if (back != null) { var b = back.GetComponent<Butt
on>(); if (b != null) b.onClick.AddListener(() => _onBack(
)); }
}
private (Image, RectTransform) BindToggle(Transform ca
rd, string rowName, Func<bool> get, Action<bool> set)
{
    Image track = null; RectTransform handle = null;
    var row = card.Find(rowName);
    if (row == null) return (null, null);
    var tr = row.Find("Track");
    if (tr == null) return (null, null);
    track = tr.GetComponent<Image>();
    var btn = tr.GetComponent<Button>();
    handle = tr.Find("Handle")?.GetComponent<RectTrans
form>();
    Action refresh = () =>
    {
        bool on = get();
        if (track != null) track.color = on ? new Colo
r(0.45f, 0.58f, 0.35f) : new Color(0.78f, 0.74f, 0.66f);
        if (handle != null) handle.anchoredPosition =
new Vector2(on ? 30f : -30f, 0f);
    };
    refresh();
    if (btn != null) btn.onClick.AddListener(() => { s
et(!get()); refresh(); });
    return (track, handle);
}
private void RefreshQuality()
{
    for (int i = 0; i < 3; i++)
    {
        bool sel = (_data.quality == i);
        if (_qualityBtns[i] == null) continue;
        if (_qualityBtns[i].image != null)
            _qualityBtns[i].image.color = sel ? new Co
lor(0.45f, 0.58f, 0.35f) : new Color(0.85f, 0.80f, 0.68f);
```

```
        var t = _qualityBtns[i].GetComponentInChildren
    <Text>();
        if (t != null) t.color = sel ? Color.white : n
    ew Color(0.35f, 0.30f, 0.22f);
      }
    }
    private void RefreshLang()
    {
      if (_langText != null) _langText.text = (_data.lan
    guage == 0) ? "简体中文    " : "English    ";
    }
}
// ===== FILE: Assets\Scripts\UI\TileView.cs =====
using System;
using System.Collections;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;
/// <summary>
/// 单张牌的视图：牌底(Body) + 牌面(Face) + 背面(Back) + 阴影(Shadow)，
/// 由 Point 容器控制 rotation.y 翻转。实现点击、选中/提示高亮、
/// 提起/放下、正反面切换、翻转动画，以及供碰撞流程编排的通用移动/缩放原语。
/// 所有实例由对象池复用（TileView.ResetView 复位状态）。
/// </summary>
public class TileView : MonoBehaviour, IPointerClickHandle
    r
{
    public string tileId;
    public int faceIndex;
    public Image bodyImage;  // 正面牌底（与 Back 互换 sprite 实现正
    反面）
    public Image faceImage;  // 牌面图案（正面显示，反面隐藏）
    public Image backImage;  // 背面牌底
    public Image shadowImage; // 阴影（提起时隐藏）
    public Transform point;  // 翻转轴容器（控制 localEulerAngles
    .y）
    public Image bodySub1, bodySub2, backSub1, backSub2; /
    / 新增子层（背面状态需同步替换图片）
    public Sprite frontSprite; // 正面牌底图（body/body_1/body_2
     在正面时使用的图）
    public Sprite backSprite;  // 等级背面图 levelN（back/back_1
    /back_2 在正面时使用的图）
    public Action<string> onClick;
    public Vector2 homePos;  // LayoutTiles 设置的原位（boardRo
    ot 局部坐标）
    private bool _selected;
    private bool _hint;
    private bool _faceUp = true;
    private bool _flipping;
    public bool wasFaceDown; // 布置时随机设为反面（取消选中时切回）
    private Color _baseColor = Color.white;
    public void SetBaseColor(Color c) { _baseColor = c; }
    public void SetSelected(bool v) { _selected = v; Updat
    eTint(); }
    public void SetHint(bool v) { _hint = v; UpdateTint();
     }
    private void UpdateTint()
    {
      if (bodyImage == null) return;
      Color c = _baseColor;
      if (_selected) c = new Color(1.0f, 0.84f, 0.40f);
     // 金色（选中）
      else if (_hint) c = new Color(0.40f, 1.0f, 1.0f);
    // 青色（提示）
      bodyImage.color = c;
      if (bodySub1 != null) bodySub1.color = c; // Body_
    1/Body_2 同步选中/提示色
```

```
    if (bodySub2 != null) bodySub2.color = c;
}
public void SetFaceSprite(Sprite tex) { if (faceImage
!= null) faceImage.sprite = tex; }
public void SetAlpha(float a)
{
    if (bodyImage != null) bodyImage.color = new Color
(bodyImage.color.r, bodyImage.color.g, bodyImage.color.b,
a);
    if (faceImage != null) faceImage.color = new Color
(faceImage.color.r, faceImage.color.g, faceImage.color.b,
a);
    if (backImage != null) backImage.color = new Color
(backImage.color.r, backImage.color.g, backImage.color.b,
a);
}
// ---------- 提起 / 放下 ----------
public void Lift() { if (shadowImage != null) shadowIm
age.enabled = false; }
public void Drop() { if (shadowImage != null) shadowIm
age.enabled = true; }
// ---------- 通用移动 / 缩放（供碰撞流程编排） ----------
public Coroutine MoveTo(Vector2 target, float dur, Act
ion onDone = null) { return StartCoroutine(TweenTo(target,
 dur, onDone)); }
public void TweenScale(float s, float dur, Action onDo
ne = null) { StartCoroutine(ScaleTo(s, dur, onDone)); }
private IEnumerator TweenTo(Vector2 target, float dur,
 Action onDone = null)
{
    var tr = GetComponent<RectTransform>();
    Vector2 from = tr.anchoredPosition;
    float t = 0f;
    while (t < dur)
    {
        t += Time.deltaTime;
        float k = Mathf.Clamp01(t / dur);
        tr.anchoredPosition = Vector2.Lerp(from, targe
t, k);
        yield return null;
    }
    tr.anchoredPosition = target;
    onDone?.Invoke();
}
private IEnumerator ScaleTo(float s, float dur, Action
 onDone = null)
{
    var tr = GetComponent<RectTransform>();
    Vector3 from = tr.localScale;
    Vector3 to = new Vector3(s, s, 1f);
    float t = 0f;
    while (t < dur)
    {
        t += Time.deltaTime;
        float k = Mathf.Clamp01(t / dur);
        tr.localScale = Vector3.Lerp(from, to, k);
        yield return null;
    }
    tr.localScale = to;
    onDone?.Invoke();
}
// ---------- 对象池复用 / 重开时复位视图状态 ----------
public void ResetView()
{
    StopAllCoroutines();
    transform.localScale = Vector3.one;
    SetAlpha(1);
```

```
    _selected = false;
    _hint = false;
    _flipping = false;
    wasFaceDown = false;
    if (bodyImage != null) bodyImage.color = _baseColo
r;
    if (bodySub1 != null) bodySub1.color = _baseColor;
    if (bodySub2 != null) bodySub2.color = _baseColor;
    if (point != null) point.localEulerAngles = Vector
3.zero;
    ToggleFace(true); // 回到正面
    if (shadowImage != null) shadowImage.enabled = tru
e;
    if (faceImage != null) faceImage.enabled = true;
}
// ---------- 正反面 ----------
// toFront=true: 正面（body/body_1/body_2=front，back/back
_1/back_2=levelN）
// toFront=false: 反面（相反）
public void ToggleFace(bool toFront)
{
    _faceUp = toFront;
    if (faceImage != null) faceImage.enabled = toFront
;
    Sprite bodyUse = toFront ? frontSprite : backSprit
e;  // 正面态用 front，反面态用 levelN
    Sprite backUse = toFront ? backSprite : frontSprit
e;  // 反面态用 front
    if (bodyImage != null) bodyImage.sprite = bodyUse;
    if (bodySub1 != null) bodySub1.sprite = bodyUse;
    if (bodySub2 != null) bodySub2.sprite = bodyUse;
    if (backImage != null) backImage.sprite = backUse;
    if (backSub1 != null) backSub1.sprite = backUse;
    if (backSub2 != null) backSub2.sprite = backUse;
}
public bool IsFaceUp() { return _faceUp; }
// ---------- 翻转（提起 -> 半程切换 -> 落下） ----------
public void Flip()
{
    if (_flipping) return;
    StartCoroutine(FlipRoutine());
}
private IEnumerator FlipRoutine()
{
    _flipping = true;
    Lift();
    yield return RotateY(0f, 90f, 0.12f);
    ToggleFace(!_faceUp);
    yield return RotateY(90f, 0f, 0.12f);
    Drop();
    _flipping = false;
}
private IEnumerator RotateY(float from, float to, floa
t dur)
{
    float t = 0f;
    while (t < dur)
    {
        t += Time.deltaTime;
        float y = Mathf.Lerp(from, to, Mathf.Clamp01(t
 / dur));
        if (point != null) point.localEulerAngles = ne
w Vector3(0f, y, 0f);
        yield return null;
    }
    if (point != null) point.localEulerAngles = new Ve
ctor3(0f, to, 0f);
```

```
    }
    // 布置关卡时直接设为反面（无动画），并标记 wasFaceDown 以便取消选中时切回
    public void SetFaceDownInitial()
    {
        ToggleFace(false);
        wasFaceDown = true;
    }
    // 翻转到正面（指定总时长，分两段各一半），完成后回调
    public void FlipToFront(float dur, Action onDone = nul
    l)
    {
        if (_faceUp) { onDone?.Invoke(); return; }
        if (_flipping) { onDone?.Invoke(); return; }
        StartCoroutine(FlipRoutineTo(true, dur, onDone));
    }
    // 翻转到反面（指定总时长，分两段各一半），完成后回调
    public void FlipToBack(float dur, Action onDone = null
    )
    {
        if (!_faceUp) { onDone?.Invoke(); return; }
        if (_flipping) { onDone?.Invoke(); return; }
        StartCoroutine(FlipRoutineTo(false, dur, onDone));
    }
    private IEnumerator FlipRoutineTo(bool toFront, float
    dur, Action onDone)
    {
        _flipping = true;
        Lift();
        float half = Mathf.Max(0.01f, dur * 0.5f);
        yield return RotateY(0f, 90f, half);
        ToggleFace(toFront);
        yield return RotateY(90f, 0f, half);
        Drop();
        _flipping = false;
        onDone?.Invoke();
    }
    public void OnPointerClick(PointerEventData eventData)
     { onClick?.Invoke(tileId); }
}
// ===== FILE: Assets\Scripts\UI\UiTools.cs =====
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// 运行时 / Editor 共用的极简 UI 构建工具。
/// 生成脚本用它把完整 UI 层级建进预制体；面板组件用它运行时填充动态列表（关卡 / 图鉴）。
/// </summary>
public static class UiTools
{
    public static Font LoadFont() { return Resources.GetBu
    iltinResource<Font>("LegacyRuntime.ttf"); }
    public static GameObject MakeGO(string name, Transform
     parent)
    {
        var go = new GameObject(name, typeof(RectTransform
    ));
        if (parent != null) go.transform.SetParent(parent,
     false);
        return go;
    }
    public static Text AddText(Transform parent, string tx
    t, int size, Color color, Font font = null)
    {
        if (font == null) font = LoadFont();
        var go = MakeGO("Text", parent);
        var t = go.AddComponent<Text>();
        t.text = txt; t.font = font; t.fontSize = size;
        t.alignment = TextAnchor.MiddleCenter; t.color = c
```

```
olor;
    var tr = go.GetComponent<RectTransform>();
    tr.anchorMin = Vector2.zero; tr.anchorMax = Vector
2.one; tr.offsetMin = Vector2.zero; tr.offsetMax = Vector2
.zero;
    return t;
}
public static Button AddButton(Transform parent, strin
g label, Font font = null)
{
    if (font == null) font = LoadFont();
    var go = MakeGO("Button", parent);
    var img = go.AddComponent<Image>();
    img.color = new Color(0.20f, 0.50f, 0.82f);
    var btn = go.AddComponent<Button>();
    btn.targetGraphic = img;
    AddText(go.transform, label, 26, Color.white, font
);
    return btn;
}
// 全屏遮罩面板（铺满父容器 + 半透明黑底 + 居中垂直布局），不含子内容
public static GameObject MakePanel(string name)
{
    var go = new GameObject(name, typeof(RectTransform
));
    var tr = go.GetComponent<RectTransform>();
    tr.anchorMin = Vector2.zero; tr.anchorMax = Vector
2.one; tr.offsetMin = Vector2.zero; tr.offsetMax = Vector2
.zero;
    var img = go.AddComponent<Image>();
    img.color = new Color(0f, 0f, 0f, 0.7f);
    var lg = go.AddComponent<VerticalLayoutGroup>();
    lg.spacing = 20; lg.childAlignment = TextAnchor.Mi
ddleCenter;
    lg.padding = new RectOffset(40, 40, 40, 40);
    return go;
}
}
```